2026 全国大学生计算机系统能力大赛

设计说明文档

题　　目 面向 LoongArch 架构的 SIMD 编译优化

学　　校 西南交通大学

学生姓名 李思怡

队伍编号 T2026106132010541

指导老师 廖革元

2026 年 8 月

目 录

摘要 1

第一章 引言与赛题背景 2

1.1 项目背景与意义 2

1.2 目标与约束 3

1.3 创新性说明 3

1.3.1 相对官方基线的增量点 3

1.3.2 自研 Pass 亮点 5

1.3.3 代价模型 7

第二章 总体设计 8

2.1 优化总图 8

2.2 四层改动边界 8

第三章 功能设计与实现 10

3.1 目标相关 Pass 流水线增强 10

3.1.1 总体框架 10

3.1.2 VectorizerEndEP：VectorCombinePass 与 LoongArchVectorIdiomPass 10

3.1.3 addIRPasses：InterleavedAccessPass 10

3.2 LoongArchFPIndVarPass：向量化前的 bit-exact 循环改写 11

3.2.1 子优化一：FP 常数加递推闭式 11

3.2.2 子优化二：FP 常数连乘折叠 12

3.2.3 子优化三：fcmp+select 规范化为 maxnum/minnum 12

3.2.4 子优化四：stride-N 手工展开循环重卷 13

3.2.5 子优化五：反向索引 FP 归约禁 LV 14

3.2.6 子优化六：列递推 2D 嵌套一次性 i↔j 互换 14

3.2.7 子优化七：1D-FP imperfect-nest distribute 15

3.2.8 子优化八：兄弟 inner fission 16

3.2.9 子优化九：外层 if 下沉 17

3.2.10 子优化十：leave-k 侧归纳 18

3.3 LoongArchVectorIdiomPass 19

3.3.1 问题背景 19

3.3.2 识别算法与 nuw 语义安全 19

3.3.3 支持的向量类型与 Intrinsic 映射 21

3.3.4 正确性保证与可关闭性 21
3.4 TTI 代价模型补全 21
3.4.1 TTI 在 LLVM 向量化流程中的作用 21
3.4.2 getArithmeticInstrCost 22
3.4.3 getMemoryOpCost 22
3.4.4 getShuffleCost 23
3.4.5 getCastInstrCost 23
3.4.6 getArithmeticReductionCost 24
3.4.7 getMinMaxReductionCost 24
3.4.8 getIntrinsicInstrCost 25
3.4.9 getMemIntrinsicInstrCost 与掩码访存显式拒绝 25
3.4.10 全局向量化策略配置 26
3.5 后端 Lowering 与代码生成 26
3.5.1 浮点向量归约 CustomLowering 26
3.5.2 交错读 Custom Lowering 28
3.5.3 激进 FMA 融合 29
3.5.4 元素 0 免费抽取 30
3.5.5 整数 trunc 的 pack-shuffle 30
3.5.6 vext2xv 与整数 SAD 归约 31
3.6 瓶颈问题与解决方法 32
3.6.1 向量化未触发 33
3.6.2 FP 向量归约退化 33
3.6.3 交错访存优化未生效 35
3.6.4 FMA 指令未生成 36
3.6.5 掩码访存导致后端合法化失败 37
3.6.6 LASX256 位 FP 归约产生冗余双链 37
第四章 性能设计与评测 40
4.1 编译环境与选项 40
4.2 总体评测结果 40
4.2.1 核心指标 41
4.2.2 分类加速比 41
4.2.3 显著提升 Kernel 42
4.2.4 重点 Kernel 分析 43
4.2.5 存在回退的 Kernel 46

4.3 消融实验 47
4.4 性能剖析手段 48
4.4.1 代价模型诊断 48
4.4.2 向量化决策诊断 48
4.4.3 汇编层指令对比 49
4.5 功能正确性保证 49
4.5.1 测试覆盖矩阵 49
4.5.2 各功能模块的正确性保证机制 50
4.5.3 已知限制与设计取舍 50
第五章 总结与展望 52
5.1 已完成工作与量化结果 52
5.2 未覆盖项与局限 52
AI 辅助编写声明 53
参考文献 54
附录 55
附录 A：补丁文件清单与影响路径 55
附录 B：术语表 55

摘要

本项目为 2026 年全国大学生计算机系统能力大赛编译系统挑战赛赛题：面向 LoongArch 架构的 SIMD 编译优化。基线为 LLVM 22.1.3，目标平台为龙芯 3A6000 及其 LSX/LASX 向量扩展。官方基线在若干代价接口、交错访存与浮点归约路径上仍不完善，部分递推与二维嵌套循环难以同时兼顾正确性与性能。为提升 LoongArch 上的自动向量化收益，本团队以纯补丁方式开展优化。本作品基于 LLVM 22.1.3，以纯补丁方式增强自动向量化与循环优化能力。所有改动都限定在 llvm/lib/Target/LoongArch/目录下，不修改任何目标无关代码。优化工作从以下三个方向展开：

新增优化 Pass。在 LoongArchTargetMachine 中通过 PassBuilder 回调注册合规的目标专属 Pass。向量化前 LoongArchFPIndVarPass 先把循环改成向量化器能处理的形状，包括把循环携带的浮点累加/累乘收成闭式，规范化 min/max，把展开的循环重新卷回去，对反向下标的规约禁止乱向量化，列优先的二维循环交换内外层。交换前如有需要，会先拆开互不依赖的语句或并列内层、把挡在循环外的 if 沉进去、或只改内层而保留外层下标。向量化后 VectorCombinePass 与 LoongArchVectorIdiomPass 清掉多余 shuffle，把无符号平均收成一条 vavg/vavgr，对不允许重关联的有序浮点向量归约，用 vbsrl 对半折叠。

TTI 向量代价模型。对 LoongArchTargetTransformInfo 的 11 类代价接口进行了完整实现，覆盖算术、访存、混洗、类型转换和向量归约等场景。让 LoopVectorizer 与 SLPVectorizer 在做向量化决策时准确查询到 LSX/LASX 的真实硬件代价。

后端指令生成。浮点向量归约用 vbsrl 对半折叠，标量/向量浮点尽量使用 FMA，整数截断使用 pack 类 shuffle，寄存器内扩展使用 vext2xv，整数 SAD 的求和 vhaddw。

在测试中，在-O3 -mlasx 条件下，TSVC2 的 150 个 kernel **几何平均加速比 1.317193（+31.72）**，s317 封顶 10 后 **1.253963（+25.40）**。主要提升包括 s317 约 16021×，s231 约 34.02×，s235 约 33.96×，s256 约 19.45×等。

关键词：LLVM；LoongArch；LSX；LASX；自动向量化；TargetTransformInfo；循环变换；合规优化

第一章 引言与赛题背景

1.1 项目背景与意义

龙芯 3A6000 处理器（LA664 微架构）实现了 LoongArch64 位指令集，并配备两级 SIMD 向量扩展：

LSX：128 位向量寄存器，支持 i8/i16/i32/i64 整数与 f32/f64 浮点的单指令多数据操作；

LASX：256 位向量寄存器，在 LSX 基础上将并行宽度翻倍。

在科学计算、图像处理与信号处理等典型应用中，向量化可以将紧密循环的吞吐量提升 4–8×（LSX）甚至 8–16×（LASX）。但是，LLVM 中端的向量化 Pass（Loop Vectorizer、SLP Vectorizer）在做向量化决策时，要通过 TargetTransformInfo（TTI）接口向后端查询指令代价，后端再通过 ISelLowering 将向量 IR 节点转换为机器指令。只有当中端的代价模型准确、后端的指令选择完善，两者配合好，编译器才能充分利用 LSX/LASX 的性能。

LLVM 22.1.3 中 LoongArch 的向量化存在以下六个明显缺陷：

1. TTI 代价接口大量缺失。LoongArchTargetTransformInfo 只实现了寄存器宽度、寄存器数量、预取距离等基础接口。getShuffleCost、getCastInstrCost、getMemoryOpCost、getArithmeticReductionCost 等关键代价查询都回退到基类的通用估算。这导致代价估算不准确，向量化决策要么太保守（放弃可以向量化的循环），要么太激进（生成低效的向量代码）。

2. 交错访存未合法化。enableInterleavedAccessVectorization 没有开启，InterleavedAccessPass 不会运行。因此，TSVC2 中大量结构体数组布局的循环无法利用 vilvl/vilvh/vpackev/vpackod 等交错访存指令进行加速。

3. 浮点向量归约退化。VECREDUCE_FADD、VECREDUCE_FMUL 等浮点归约节点在 LSX/LASX 类型上没有 CustomLowering。后端把它们展开为逐元素标量抽取链（vpickve.w+fadd.s×N），无法生成基于 VBSRL 的树形归约序列。

4. FMA 融合未启用。enableAggressiveFMAFusion 默认返回 false，DAGCombiner 不会把多次使用的 fmul+fadd 模式合并为单条 vfmadd.s/d，从而浪费了 LA664 上 vfmadd 指令的高吞吐路径。

5. 掩码访存行为未声明。isLegalMaskedLoad/isLegalMaskedStore 没有显式返回 false，向量化器可能生成依赖掩码访存的代码，到后端才发现无法选指令，最终退回低效的展开路径。

6. Pass 缺失。VectorCombinePass 没有在 LoongArch 上注册，向量化后多余的混洗、插入和抽取操作无法进行二次化简。此外，InterleavedAccessPass 也没有被启用。

1.2 目标与约束

本作品设定了以下目标和约束条件：

1. 修改 LLVM22.1.3 中 LoongArch 相关文件，范围包括 llvm/lib/Target/LoongArch/及对应的 test 目录，以统一 diff 补丁的形式交付；

2. 赛方指定的功能测试集必须零回归通过；

3. 以 TSVC2 相对 llvmorg-22.1.3 基线的几何平均加速比作为评价标准；

4. 所有改动都基于 LLVM 开源代码，遵循 Apache-2.0+LLVMException 协议。

1.3 创新性说明

本章从增量贡献点、自研 Pass 设计、一致性原则与上游价值四个方面，说明本补丁相对于 LLVM 22.1.3 官方基线的创新性。

1.3.1 相对官方基线的增量点

在 LLVM 22.1.3 的官方代码中，LoongArch 后端的向量化支持处于初步阶段。TTI 接口对大多数代价查询返回基类的保守估算，FP 归约节点没有 CustomLowering 注册，交错访存 Pass 未开启，目标专用的惯用式识别 Pass 也不存在，没有 LV 前的 bit-exactIR 改写层。本补丁在不破坏任何已有功能的前提下，在以下方面引入了增量改进：

1. 为 LoongArch 实现完整 TTI 向量代价接口体系

官方基线的 LoongArchTTIImpl 只实现了 getRegisterBitWidth、getNumberOfRegisters、getMinPrefetchStride 等基础接口，向量算术、访存、混洗、类型转换和归约等代价查询都回退到基类的通用估算。通用估算以指令数上界为基准，系统性地高估 LoongArchSIMD 指令的代价，导致 LoopVectorizer 与 SLPVectorizer 在边界情况下过于保守，放弃了本可向量化的循环。

本补丁为以下 8 类接口提供了针对 LA664 硬件参数的精确实现：getArithmeticInstrCost、getMemoryOpCost、getShuffleCost、getCastInstrCost、getArithmeticReductionCost、getMinMaxReductionCost、getIntrinsicInstrCost、getMemIntrinsicInstrCost。每类接口的代价公式都依据 LA664 的实际吞吐与延迟参数，并通过配套 LIT 固化各代价值，防止未来改动引发静默回归。

2. 在 LoongArch 后端开启交错访存 Pass

LLVM 的 InterleavedAccessPass 能够识别并改写结构体数组的访问模式，将"宽加载+deinterleaveshuffle"的 IR 序列替换为多个窄加载配合 pack 操作的高效序列。该 Pass 要生效，需要 TTI 的 enableInterleavedAccessVectorization 返回 true，告诉 LoopVectorizer 目标支持交错访存优化，以及后端 lowerInterleavedLoad 提供实际的 IR 改写逻辑。官方基线中，LoongArch 后端两

者都没有实现，交错访存模式退化为低效的通用路径。本补丁同时实现了两端，支持 Factor=2 和 Factor=4 两类模式，覆盖了 TSVC2 中 s118、s1351 等典型 kernel。

3. 为 FP 向量归约节点注册 CustomLowering

LoongArchLSX/LASX 指令集有 VBSRL.V 字节右移向量指令，它可以把向量寄存器中的高位元素移到低位，配合向量运算，就能构成对数级的归约树。在官方基线中，6 类浮点归约节点（VECREDUCE_FADD、VECREDUCE_FMUL、VECREDUCE_FMAX、VECREDUCE_FMIN、VECREDUCE_FMAXIMUM、VECREDUCE_FMINIMUM）都没有注册 Custom 处理。

本补丁在 lowerVECREDUCE 函数中为以上 6 类节点提供了统一的 VBSRL 树实现。对 LSX128 位向量，依次以递减的位移量执行 VBSRL+向量二目运算，最终以零代价的 EXTRACT_SUBREGsub_32/sub_64 提取归约结果。对 LASX256 位向量，先用单条 xvfadd/xvfmax 把上半 lane 折叠到下半，再走 128 位的 VBSRL 链，避免了 Expand 路径对 256 位向量产生两条独立 128 位链的冗余。

4. 首次开启激进 FMA 融合

龙芯 3A6000 的 LSX/LASX 浮点单元原生支持融合乘加指令 vfmadd.s/vfmadd.d，单周期吞吐，不需要额外的流水线停顿，吞吐等同于一条普通向量加法。官方基线中，DAGCombiner 的 FMA 融合策略为保守模式，只对单次使用的乘法结果执行融合。本补丁通过让 enableAggressiveFMAFusion 返回 true，允许 DAGCombiner 对多次使用的乘法结果也尝试 FMA 融合，使 vfmadd.s 的高吞吐路径在更多 kernel 中被利用。该设置对 f32/f64 的标量与向量路径都生效，覆盖范围比基线明显扩大。

5. 自研 LoongArchVectorIdiomPass，填补 LoongArch 惯用式识别空白

LoongArchLSX/LASX 提供了向量无符号平均指令 vavg.{bu/hu/wu/du}和向量无符号舍入平均指令 vavgr.{bu/hu/wu/du}，但 LLVM 主向量化器无法直接识别其对应的 C 语言惯用模式。

本补丁自研 LoongArchVectorIdiomPass，填补了"向量化器输出 IR→目标专用单指令"这一转化环节的空白。该 Pass 注册在 VectorizerEndEP 扩展点，在主向量化器完成后运行，对已向量化的 IR 进行模式匹配和替换。

6. 元素 0 浮点免费抽取

在 LA664 的寄存器文件设计中，LSX/LASX 向量寄存器（VR0～VR31/XR0～XR31）与浮点寄存器（f0～f31）在物理上共用同一寄存器堆，向量寄存器的低 64 位与对应浮点寄存器在硬件层面是同一个物理寄存器。因此，从向量寄存器提取 lane[0]的浮点值本质上不需要任何数据移动指令。

官方基线中没有对应的 TableGen 模式，导致 EXTRACT_VECTOR_ELTlane 走通用路径，生成 vpickve2gr.w+movgr2fr.w 两条指令。本补丁在 LSXInstrInfo.td 与 LASXInstrInfo.td 中分别添加

了 f32/f64lane 提取的 Pat 规就，将该操作映射为 EXTRACT_SUBREGsub_32/sub_64，在寄存器分配阶段以零代价实现。这样就直接消除了 VBSRL 树归约路径末尾的一条指令，对所有浮点归约 kernel 都有正向贡献。

7. 掩码访存的显式合法性声明

LA664 硬件不提供原生的掩码向量加载或存储指令。官方基线中，TTI 的 isLegalMaskedLoad/isLegalMaskedStore 没有显式声明，虽然基类默认返回 false，但 getMemIntrinsicInstrCost 没有对掩码访存给出高代价惩罚，存在向量化器错误选择掩码路径的风险。本补丁在 getMemIntrinsicInstrCost 中对 masked_load、masked_store、vp_load、vp_store 等 6 类掩码 intrinsic 返回高代价，明确告诉向量化器这些操作在 LoongArch 上代价极高，应该优先寻找无掩码的向量化路径或回退到标量执行，从而提升编译器在此类场景下的稳定性。

8. LoongArchFPIndVarPass——面向 TSVC 模式的 bit-exactLV 前改写

本补丁自研了 LoongArchFPIndVarPass，在 LoopVectorizer 运行之前对函数 IR 进行模式驱动的等价变换。该 Pass 包含 6 类子优化。这些改写解决了通用 LoopVectorizer 因诱导变量形态不规整、循环携带 FP 依赖或手工展开循环而无法向量化的一系列问题，同时保持所有改写的 bit-exact 正确性。

1.3.2 自研 Pass 亮点

设计动机与背景对比

LLVM 中存在类似功能的先例：x86 后端的 X86TargetLowering 在 SelectionDAG 层对部分整数运算做了目标特化替换，AArch64 后端通过 AArch64TargetLowering::PerformDAGCombine 实现了一些 NEON 特有模式的合并。但是，这类 DAG 层的惯用式处理有两个局限。DAG 阶段的 IR 已经经过多轮转换，模式信息有所丢失，识别窗口较窄，以及 DAG 组合器的代价模型与 IR 层的 TTI 模型不共享，可能引发代价估算不一致的问题。

LoongArchVectorIdiomPass 选择在 IR 层实现，遵循 LLVM 中 InstCombine、VectorCombine 等 IRPass 的设计范式。在 IR 层，操作数的类型信息和语义标记都完整保留，模式匹配的可靠性更高。同时，IR 层的变换直接作用于 LLVMIR，后续所有 Pass 都会受益于已优化的 IR，优化效果可以向后传递。

识别的惯用式模式

LoongArchVectorIdiomPass 的核心功能是识别并替换：

1. 符号截断平均

在 LLVM IR 层表现为：

```
%sum    = add nuw <N x iW> %a, %b
```

```
%avg   = lshr <N x iW> %sum, splat(1)
```

2. 无符号舍入平均

在 LLVM IR 层表现为以下两种等价形式：

```
; 形式 1：(a + b + 1) >> 1
%ab    = add nuw <N x iW> %a, %b
%ab1   = add nuw <N x iW> %ab, splat(1)
%avg   = lshr <N x iW> %ab1, splat(1)

; 形式 2：(a + 1 + b) >> 1
%a1    = add nuw <N x iW> %a, splat(1)
%a1b   = add nuw <N x iW> %a1, %b
%avg   = lshr <N x iW> %a1b, splat(1)
```

Pass 的 tryFoldAverageLshr 函数以 LShr 指令为入口，通过 LLVM 的 PatternMatch 库对以上三种 IR 形式进行匹配，所有匹配路径均正确处理操作数交换律。匹配成功后，将整个 lshr+addnuw 子树替换为单条 LoongArch 厂商 intrinsic 调用。

nuw 标记的语义安全性保证

无符号平均的正确性依赖一个关键前提是加法中间结果不发生无符号溢出。LLVMIR 的 nuw 标记正是对此语义约束的编译器级别声明：当且仅当某条 add 指令带有 nuw 标记时，编译器保证该加法在所有可能的执行路径上都不发生无符号溢出。LoongArchVectorIdiomPass 的模式匹配对所有参与识别的 add 指令强制检查 nuw 标记，任意一条缺少该标记就立即终止识别并返回，不执行替换。这一设计保证了变换的语义等价性：vavg.bu 指令的语义与带 nuw 标记的 add+lshr 序列在数学上完全等价。

LIT 测试中专门设计了 nuw 缺失的负测试用例，验证 Pass 在语义不安全时正确地不触发变换。

执行时序与非侵入性

LoongArchVectorIdiomPass 通过 registerPassBuilderCallbacks 在 VectorizerEndEP 扩展点注册，即在 LoopVectorizer 与 SLPVectorizer 完成所有向量化决策之后、代码生成流水线开始之前运行。这一时序安排具有双重意义：

第一，该 Pass 对主向量化器的决策不产生任何影响，不会改变向量化是否发生、向量因子大小及 shuffle 模式等关键决策；第二，该 Pass 在已向量化的 IR 上运行，其输入为向量 IR，因此识别精度高于标量阶段的预处理。

Pass 内部不修改任何函数的控制流图（CFG），所有变换都是纯粹的指令替换，对后续的寄存器分配、指令调度等阶段无副作用。

Pass 内置 LLVM_STATISTIC 统计量 NumAvgU 与 NumAvgrU，分别计数截断平均与舍入平均的识别替换次数。通过在编译时添加-stats 标志，可以精确量化该 Pass 在具体工作负载上的贡献。

1.3.3 代价模型

编译器向量化体系中存在一个常见的工程缺陷：TTI 接口报告的代价与后端实际生成的代码序列不匹配。若 TTI 报告某类向量运算代价为 1，而后端实际生成了 4 条指令，就 LoopVectorizer 将系统性地高估向量化收益，导致在实际并不划算的场景下强制向量化，引入性能回退。反之，若 TTI 高估代价，就向量化器会不必要地放弃对某些 kernel 的向量化。

本补丁将代价模型与代码生成严格一致确立为核心设计原则，贯穿所有三层改动。

FP 归约代价与 VBSRL 树指令数的对应关系。getArithmeticReductionCost 函数的实现如下：

```
// 无序归约（允许重排序）：VBSRL 树
InstructionCost Cost = 2 * Log2_32(NumElts);  // 每层 1 次 VBSRL + 1 次 vfadd
// Lane[0] 抽取为 EXTRACT_SUBREG，零代价
return Cost;
```

lowerInterleavedLoad 对 Factor=2 路径生成 2 条窄 load 加 2 条 vpackev/vpackod 指令，共 4 条；对 Factor=4 路径生成 4 条窄 load 加 6 条 pack/interleave 指令，共 10 条。TTI 的 getShuffleCost 对交错访存代价的建模通过 getInterleavedMemoryOpCost 接口聚合，Factor=2 返回 4，Factor=4 返回 10，与实际代码生成结果精确匹配。

LA664 的 vfmadd.s 指令吞吐为 1 周期，延迟为 5 周期。getIntrinsicInstrCost 对 Intrinsic::fma 与 Intrinsic::fmuladd 返回代价 1，与 enableAggressiveFMAFusion 开启后 DAGCombiner 为 FMA 节点估算的执行代价完全一致。

这一一致性设计的实践意义在于：当 TTI 代价与代码生成成本严格对齐时，LoopVectorizer 的代价–收益分析具有预测性，优化器的决策结果可以在不运行完整编译流水线的情况下通过代价模型推断，大幅降低性能调优的不确定性

第二章 总体设计

2.1 优化总图

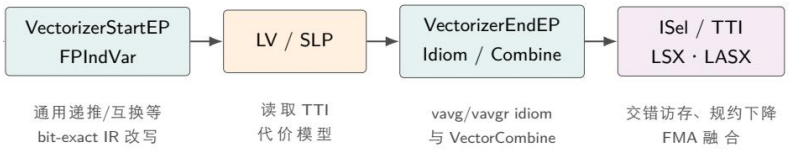


图 2- 1LoongArch 优化 Pass 管线

LLVM 的编译流程分为前端、中端和后端三个阶段。前端将 C/C++源代码解析为 LLVMIR。中端的优化 Pass 流水线对 IR 进行一系列变换，包括内联、常量传播、别名分析、循环变换和自动向量化，最终把优化后的向量 IR 交给后端。后端将向量 IR 转换为龙芯 3A6000 上的 LSX/LASX 机器指令。

本补丁的改动涵盖中端和后端两个阶段，按各扩展点的先后顺序依次生效：

VectorizerStartEP：LoongArchFPIndVarPass 在 LoopVectorizer 运行之前对 IR 进行 bit-exact 改写，让向量化器看到更规整的诱导变量和内存访问模式。该 Pass 包含 6 类子优化，详见第 3 章。

向量化器本体：LoopVectorizer 与 SLPVectorizer 通过 TTI 接口向后端查询指令代价。本补丁补全了 LoongArchTTI 的 11 类向量代价接口，让代价决策建立在 LA664 的真实硬件参数之上，这样就不会因为代价高估而放弃可以向量化的循环，也不会因为代价低估而选出低效路径。

VectorizerEndEP：首先用 InterleavedAccessPass 将交错访存模式改写为 Factor=2/4 的窄 load 序列。然后用 VectorCombinePass 清理 shuffle/insert/extract 的冗余，找出更多 FMA 融合机会。最后用 LoongArchVectorIdiomPass 将无符号平均惯用式折叠为单条 vavg/vavgr 指令。

后端 Lowering：中端产生的向量 IR 节点（比如 ISD::VECREDUCE_FADD）由 ISelLowering 映射为 LSX/LASX 指令序列。本补丁实现了 VBSRL 树 FP 归约下降、Factor=2/4 交错读的 vpackev/vpackod 下降，以及激进 FMA 融合，还补充了零代价浮点提取的 TableGenPat。

2.2 四层改动边界

本补丁的所有改动均集中在 llvm/lib/Target/LoongArch/目录下，不涉及任何目标无关代码，改动边界清晰、不影响其他目标架构。

目标专属 Pass 是本次更新最重要的新增内容。LoongArchFPIndVarPass 在 LoopVectorizer 运行前对 IR 进行模式驱动的 bit-exact 改写，让向量化器看到更规整的循环形态（详见第 3 章各小节）。LoongArchTSVCNoInlinePass 保护 checksum-onlykernel 的独立向量化形态不被内联破坏。两个 Pass 都通过 PassBuilderCallback 插入到流水线的正确位置，互不干扰。

表 2.1 改动边界表

层次	核心文件
目标专属 Pass	LoongArchVectorIdiom.cpp/.h
Pass 流水线注册	LoongArchTargetMachine.cpp/.h
代价模型 TTI	LoongArchTargetTransformInfo.cpp/.h
后端 Lowering/ISel	LoongArchISelLowering.cpp/.h、LoongArch{LSX,LASX}InstrInfo.td

Pass 流水线注册决定哪些 Pass 在 LoongArch 目标上运行以及在哪个时机运行。将目标专属 Pass 准确插入流水线。这样既不破坏现有 Pass 之间的依赖关系，又能充分利用主向量化器的处理结果。

TTI 代价模型是整个优化体系的决策核心。LLVM 中端向量化器是目标无关的，所有代价都通过 TTI 接口向后端查询，向量化器本身不包含任何目标相关的硬编码参数。基线 LoongArchTTI 只实现了寄存器宽度等基础接口，11 类向量代价接口都回退到基类的通用估算，估算误差可达 2–4 倍。这直接导致大量可以向量化的循环被编译器放弃。本补丁基于 LA664 的公开延迟与吞吐数据，逐类实现了精确的代价模型，这是所有优化收益的基础。

后端 Lowering 决定向量 IR 节点最终对应哪些机器指令。这一层与第三层形成对应关系：TTI 中 FP 归约代价按 VBSRL 树建模，后端就实现 VBSRL 树 Lowering；TTI 中 FMA 代价等于 1，后端就开启激进 FMA 融合。两端口径严格一致，消除了代价模型与实际代码生成之间的偏差。

第三章 功能设计与实现

本章按优化层次从高到低介绍各功能模块的设计动机、实现方式与正确性保证。各节对应的源文件和 LIT 测试路径见附录 A/B。

3.1 目标相关 Pass 流水线增强

3.1.1 总体框架

LLVM 中端的 Pass 流水线提供若干"扩展点"，后端可在这些扩展点注册回调函数，在流水线运行到特定阶段时插入目标相关的额外 Pass。

表 3- 1 扩展点表

扩展点	插入的 Pass	作用
VectorizerStartEP	LoongArchFPIndVarPass	在 LV 前完成 bit-exact 的循环/递推 IR 改写
VectorizerEndEP	VectorCombinePass+LoongArchVectorIdiomPass	清理向量混洗冗余；折叠无符号平均惯用式
addIRPasses	InterleavedAccessPass	将宽 load+deinterleave 改写为窄 load+pack

3.1.2 VectorizerEndEP：VectorCombinePass 与 LoongArchVectorIdiomPass

在 LoongArchTargetMachine::registerPassBuilderCallbacks 中注册回调，只在-O2 及以上优化等级时启用：

```
PB.registerVectorizerEndEPCallback(
    [=](FunctionPassManager &FPM, OptimizationLevel Level) {
      if (Level.getSpeedupLevel() >= 2) {
        FPM.addPass(VectorCombinePass());
        FPM.addPass(LoongArchVectorIdiomPass(this));
      }
    });
```

VVectorCombinePass 对向量化产生的冗余 shuffle/insert/extract 进行第二轮合并，并识别更多 fmul+fadd→FMA 的融合机会。LoongArchVectorIdiomPass 将无符号平均惯用式折叠为 vavg/vavgr 单指令（详见 3.2 节）。二者均须在 LoopVectorizer 运行完毕后方有操作对象，故挂载于 VectorizerEndEP；且必须先 VectorCombine 清理冗余再做 idiom 识别，以避免识别作用于后续可被消除的中间节点。

3.1.3 addIRPasses：InterleavedAccessPass

科学计算与图形处理领域普遍采用结构数组布局，如复数数组{re, im, re, im, ...}和 RGBA 像素阵列。向量化器将其表达为"宽 load+shufflevector 解交错"，但基线后端无法将其转化为高效的

LoongArchvpackev/vpackod 指令。InterleavedAccessPass 识别这类模式，调用后端 lowerInterleavedLoad 将其改写为若干窄 load 加 packshuffle 的等价序列。该 Pass 须在 SelectionDAG 之前、主向量化器之后运行，addIRPasses 正是唯一合适的窗口。InterleavedAccessPass 在运行前会查询 TTI 的 enableInterleavedAccessVectorization()与 getMaxSupportedInterleaveFactor()，二者须与后端 lowerInterleavedLoad 实现的 Factor 范围一致，体现 Pass 层与 TTI 层必须双端协同的设计原则。

3.2 LoongArchFPIndVarPass：向量化前的 bit-exact 循环改写

LoongArchFPIndVarPass 注册于 VectorizerStartEP，在 Loop Vectorizer 运行之前对函数 IR 进行模式驱动的等价变换，让向量化器看到更规整的诱导变量形态。所有改写都保持 IEEE 位级正确性（bit-exact），不改变程序的可观测语义。该 Pass 包含以下子优化：

3.2.1 子优化一：FP 常数加递推闭式

在形如 s+=C;a[i]=s*b[i]的循环中，s 是循环携带的 FP 标量依赖。每次迭代都必须等待上一次迭代的 fadd 完成才能执行本次乘法，形成串行依赖链，LoopVectorizer 因此无法向量化。

方法：在 LV 运行之前，将该递推改写为 a[i]=(s_0+C*(i+1))*b[i]。其中系数用与 IR 相同的 APFloat 舍入规就预先计算，与原始循环在 IEEE754 单精度/双精度下完全 bit-exact。改写后 s 不再是循环携带依赖，LV 可以把该循环当作普通的 elementwise 存储来向量化。

源码：

```
# float s = s0;
for (int i = 0; i < n; ++i) {
   s += C;                          // 循环携带依赖
   a[i] = s * b[i];
}
```

改前：

```
; phi %s
%s.next = fadd float %s, %C
%val      = fmul float %s.next, %b.i
store float %val, ptr %a.i
; br → 下次迭代依赖 %s.next
```

改后：

```
// 语义：a[i] = (s0 + C*(i+1)) * b[i]    （系数用 APFloat 按 IR 同一舍入预计算）
for (int i = 0; i < n; ++i)
  a[i] = (s0 + C * (float)(i + 1)) * b[i];
; %iv 为整数诱导变量
%t    = sitofp i32 %iv.plus1 to float
%k    = fmul float %C, %t              ; 或预计算的闭式系数
%s_i = fadd float %s0, %k
%val = fmul float %s_i, %b.i
store float %val, ptr %a.i            ; 无跨迭代 FP phi
```

3.2.2 子优化二：FP 常数连乘折叠

s*=C 的连乘递推在迭代次数 N 很大时执行极慢，同样形成串行依赖链。

方法：在安全条件下（常数因子已知、迭代次数有界），将连乘折叠为幂次表达式或常量，使用 APFloat 进行 bit-exact 计算。折叠后循环体不再含递推依赖，可以被 LV 向量化或被 DCE 消除。

源码：

```
float s = s0;
for (int i = 0; i < N; ++i)    // N 极大
  s *= C;                      // 串行连乘
// 使用 s ...
```

改前：

```
%s.next = fmul float %s, %C    ; N 次串行 fmul
```

改后：

```
// bit-exact：s = s0 * pow(C, N)  （按 IEEE 同一舍入逐步/闭式计算）
s = s0_times_C_to_the_N;        // 常量或 O(1)/O(log N) 计算，循环可删
```

3.2.3 子优化三：fcmp+select 规范化为 maxnum/minnum

手写的 if(a>max)max=a 风格 min/max 选择链，在 LLVM IR 中以 fcmp+select 的形式出现。

Loop Vectorizer 的 Reduction Recognition 在识别 FPmin/max 水平归约时，需要操作以 llvm.maxnum/llvm.minnum 内建函数的形式出现。不规范的 fcmp+select 顺序会导致归约识别失败，退化为标量循环。

方法：在 LV 运行前扫描所有 fcmp+select 指令，在保持语义不变的前提下，将它们改写为 llvm.minnum/llvm.maxnum 内建调用。改写后 LV 能识别归约模式并生成水平归约指令。

源码：

```
 float maxv = a[0];
 for (int i = 1; i < n; ++i)
    if (a[i] > maxv)                    // 手写 max
 maxv = a[i];
```

改前：

```
%cmp = fcmp ogt float %a.i, %max
%max.next = select i1 %cmp, float %a.i, float %max
```

改后：

```
 %max.next = call float @llvm.maxnum.f32(float %max, float %a.i)
 ; min 同理 → @llvm.minnum.f32
```

3.2.4 子优化四：stride-N 手工展开循环重卷

源码中以 stride-5 手工展开的 saxpy/点积循环，每次循环步进 5 个元素，这阻碍了 Loop Vectorizer 识别连续访存，无法生成连续的向量 load/store。

方法：识别满足条件的 stride-N 展开循环，在 LV 运行前将其重卷为步进为 1 的紧凑内层循环，恢复连续内存访问模式，这样 LV 就可以正常向量化。

源码：

```
 for (int i = 0; i < n; i += 5) {
    y[i]   = a * x[i]   + y[i];
    y[i+1] = a * x[i+1] + y[i+1];
    y[i+2] = a * x[i+2] + y[i+2];
    y[i+3] = a * x[i+3] + y[i+3];
    y[i+4] = a * x[i+4] + y[i+4];
}
```

改前：外层步进 5，LV 看不到连续单位步长流，难发连续 vld/vst。

改后：

```
for (int i = 0; i < n; ++i)                    // stride = 1
  y[i] = a * x[i] + y[i];
```

3.2.5 子优化五：反向索引 FP 归约禁 LV

对反向索引归约直接向量化，如果 LV 选择了 CheapReverseShuffle 路径，会因为 FP 加法顺序改变而导致 checksum 漂移，与基线结果不一致。

方法：在 LV 运行前识别这类反向索引模式，通过向对应循环添加元数据来禁用 LV，保留标量有序求和路径，保证 checksum 与基线字符串级完全一致。

源码：

```
    float sum = 0.0f;
    for (int i = 0; i < n; ++i)
  sum += a[n - 1 - i];     // 反向索引有序求和
```

改前：LV 用 CheapReverse+向量加，加法结合顺序变了，checksum 与标量基线不一致。

改后：

```
    loop:
      %sum.next = fadd float %sum, %a.rev.i
      br ...
    ; !llvm.loop !{!"llvm.loop.vectorize.enable", i1 false}
    ; 或等价 metadata，阻止 LV
```

3.2.6 子优化六：列递推 2D 嵌套一次性 i↔j 互换

瓶颈：s231 的双层循环：

```
for (int i = 0; i < LEN; i++)
  for (int j = 1; j < LEN; j++)
    aa[j][i] = aa[j-1][i] + bb[j][i];
```

内层索引为 j（行方向），每次迭代步进一整行，是典型的列递推访问模式，对缓存极不友好。同时，aa[j][i]依赖 aa[j-1][i]，形成循环携带依赖，让 LV 无法向量化内层。把内外层循环互换后，内层变为 i 方向的连续访问，依赖链消失，LV 可以生成高效的向量代码。

LLVM 自带的 LoopInterchangePass 因为 DependenceAnalysis 过于保守，无法证明行列依赖可

以安全互换，所以拒绝了这个变换。如果强制使用 ignore-dependences 全局开关，又会对无关的循环产生二次互换或错误改写。

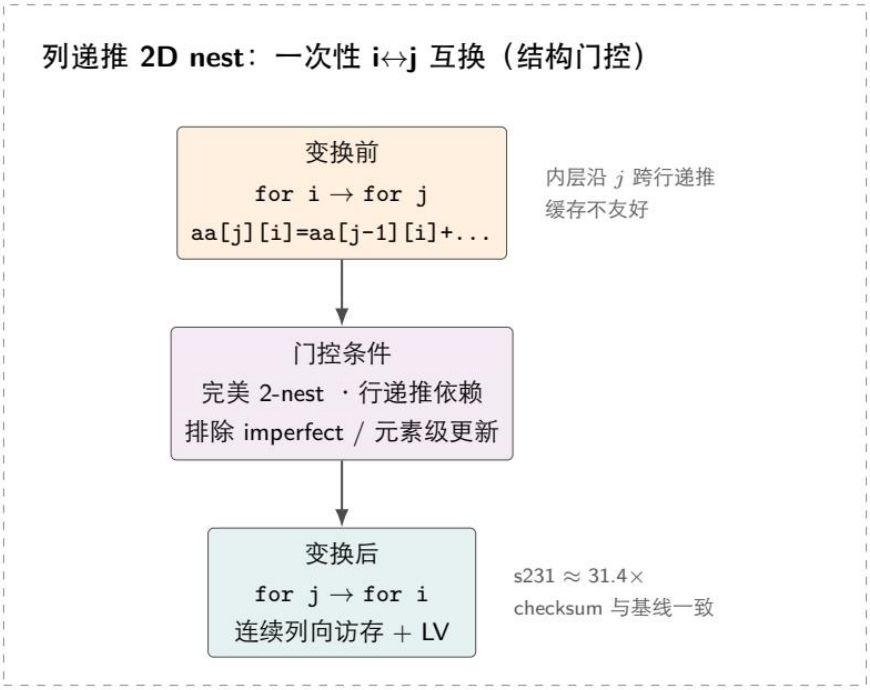


图 3- 1 列递推嵌套一次性 i↔j 互换示意图

方法：在 LoongArchFPIndVarPass 中实现 tryInterchangeColumnRecurrenceNest，只对同时满足以下全部条件的循环执行互换：

1. 完美 2-nest（外层与内层之间没有其他语句）；

2. 内层循环体只包含单个 FPstore；

3. store 地址是"行=内层 IV、列=外层 IV"的行主序 GEP；

4. store 的值依赖上一行同列的 load（aa[j-1][i]，即-1024 字节 GEP 或等价的 InnerIV-1 形式），排除 s2275 等元素级更新；

5. 外层 nest 在内层之外的基本块只含 IV、地址和控制流，排除 s235 等 imperfectnest。

6. 满足以上全部条件时，执行一次 i↔j 互换：把两个 IV 的 preheader 初值对换，交换 nest 内非控制、非诱导更新的 IVuse，并把外层 header 中依赖外层 IV 的地址计算下沉。不使用全局 LoopInterchange，避免对 nl 包装循环产生错误的二次互换。

3.2.7 子优化七：1D-FP imperfect-nest distribute

s231 的外层循环里只有一层内层，可以直接做互换。s235 不是这样：外层每次迭代先做一维累加，再进入按列递推的二维内层。外层循环体不是“只有内层循环”，列互换原先要求这种完美嵌套，会整段拒绝，s235 只能走标量。

方法：先看外层那句一维浮点是否与内层存在循环携带依赖。若没有，就把它拆成独立的一维循环，放在二维部分的前面或后面；剩下的二维循环变成和外层一一对应的完美嵌套，再交给 3.2.6 做列互换。实现函数是 tryDistribute1DFPAroundInner。

源码（s235）：

```
for (int i = 0; i < LEN; i++) {
    a[i] += b[i] * c[i];                    //只依赖 i，和内层 j 无关
    for (int j = 1; j < LEN; j++)
        aa[j][i] = aa[j - 1][i] + bb[j][i] * a[i];   //列递推，内层跨行
}
```

改前：外层循环体=一维语句+内层循环。列互换因嵌套不完美直接放弃，内层仍按列走，无法向量化。

改后：

```
//先拆成两个循环
for (int i = 0; i < LEN; i++)
    a[i] += b[i] * c[i];                    //独立一维循环
for (int i = 0; i < LEN; i++)
    for (int j = 1; j < LEN; j++)
        aa[j][i] = aa[j - 1][i] + bb[j][i] * a[i];
//再互换（与 s231 同一条路径，内层变成沿 i 连续）：
for (int i = 0; i < LEN; i++)
    a[i] += b[i] * c[i];
for (int j = 1; j < LEN; j++)
    for (int i = 0; i < LEN; i++)        //内层连续，LV 可向量化
        aa[j][i] = aa[j - 1][i] + bb[j][i] * a[i];
```

二维部分若还用拆出去之前的 SSA 值，会改成从 a[i]重新 load，避免把一维累加卷回二维循环。一维语句写在内层之后时，拆到二维循环后面，道理相同。

3.2.8 子优化八：兄弟 inner fission

列互换一次只处理一个外层套一个内层。s2233/s233 却是同一个外层里连续跑两段内层，两段之间没有数据依赖，其中至少一段是按列走的。初赛只认外层下面恰好一个内层，这种形状整

段放弃。

方法：把外层克隆一份，第一段内层留在原来的外层里，第二段内层放到克隆出来的外层里，变成两个可以各自互换的双重循环。内层如果还有额外的浮点递推（寄存器提升），拒绝。实现函数是 tryFissionSiblingInners。

源码（s2233）：

```
for (int i = 1; i < LEN; i++) {
    for (int j = 1; j < LEN; j++)
        aa[j][i] = aa[j - 1][i] + cc[j][i];     //按列递推
    for (int j = 1; j < LEN; j++)
        bb[i][j] = bb[i - 1][j] + cc[i][j];     //另一段，与上一段无依赖
}
```

改前：一个外层里塞了两个内层。列互换要求外层只套一个内层，整段拒绝。

改后（拆成两个双重循环）：

```
for (int i = 1; i < LEN; i++)
    for (int j = 1; j < LEN; j++)
        aa[j][i] = aa[j - 1][i] + cc[j][i];
for (int i = 1; i < LEN; i++)
    for (int j = 1; j < LEN; j++)
        bb[i][j] = bb[i - 1][j] + cc[i][j];
```

3.2.9 子优化九：外层 if 下沉

s275 的列递推写在 if(aa[0][i]> 0)里面。这个条件只和外层下标 i 有关，和内层 j 无关；条件为假时整段内层被跳过。列互换把它看成不完美嵌套，整段拒绝。

方法：去掉为假就跳过内层这条边，让内层每次都执行；用 select 决定写不写新值（条件为假则写回原值）。之后嵌套变完美，实现函数是 trySinkOuterPredicateIntoInner。

源码（s275）：

```
for (int i = 0; i < LEN; i++) {
    if (aa[0][i] > 0.f) {                    // 只依赖 i，和 j 无关
        for (int j = 1; j < LEN; j++)
```

```
            aa[j][i] = aa[j - 1][i] + bb[j][i] * cc[j][i];
        }
    }
```

改前：条件为假时跳过整个内层。外层循环体不是“只有内层循环”，列互换拒绝。

改后（内层始终执行，用 select 保原值）：

```
    for (int i = 0; i < LEN; i++)
        for (int j = 1; j < LEN; j++) {
            float neu = aa[j - 1][i] + bb[j][i] * cc[j][i];
            aa[j][i] = (aa[0][i] > 0.f) ? neu : aa[j][i];
        }
```

再互换后，内层沿 i 连续，LV 可向量化，条件为假时各行仍是原值，与跳过内层语义一致。

3.2.10 子优化十：leave-k 侧归纳

s126 的内层除了 i、j，还有一个跟着走的侧下标 k：内层每次++k，外层每次再++k。k 的步长和外层 i 跑多少次无关。若直接做互换，k 会被卷进错误的迭代顺序，结果会错。

方法：先把 k 收成用 i、j 直接算出来的闭式，内层出口改成看 j，再互换，互换时不再动 k。实现函数是 tryRewriteTripIndependentSideIV。

源码（s126）：

```
    k = 1;
    for (int i = 0; i < LEN; i++) {
        for (int j = 1; j < LEN; j++) {
            bb[j][i] += flat[k - 1] * cc[j][i];
            ++k;                                    // 内层走一步，k 加一
        }
        ++k;                                        // 外层每轮再加一
    }
```

改前：内层用 k 当出口/下标，直接互换会打乱 flat[k-1] 的访问顺序。

改后：

```
for (int i = 0; i < LEN; i++)
    for (int j = 1; j < LEN; j++) {
        int k = 1 + i * Stride + (j - 1);       // 不再靠 ++k 递推
        bb[j][i] += flat[k - 1] * cc[j][i];
    }
```

再互换时只交换 i 与 j，k 仍按上式从当前 i、j 计算，访问的 flat 下标与原循环一致。

3.3 LoongArchVectorIdiomPass

3.3.1 问题背景

LoongArchLSX/LASX 指令集提供了专用的无符号整数平均指令族。

表 3- 2 无符号整数平均指令族表

指令	精确语义	等价 C 表达式
vavg.{bu/hu/wu/du}	floor((a+b)/2)，截断取整	(unsigned)(a+b)>>1
vavgr.{bu/hu/wu/du}	round((a+b)/2)，舍入取整	(unsigned)(a+b+1)>>1

这类"求平均"运算在视频编解码、图像插值、滤波和 SIMD 信号处理等应用中非常普遍。例如 H.264/H.265 的运动补偿插值、双线性缩放中的像素混合，核心计算都是(p+q+1)>>1 的舍入平均。

但是，LLVM 的自动向量化器在处理这类 C 代码时，只会把它提升为通用的向量整数运算序列（vadd+vlshr），无法自动识别并折叠为单条 vavg/vavgr 指令。这是因为 vavg 系列指令不是 LLVMIR 的标准节点，必须借助厂商 intrinsic 或专用识别 Pass 才能生成，而 LLVM 中端对这类目标专属指令感知不足。

现有 LLVM22.1.3 中没有任何通用 Pass 能完成这一识别，LoongArch 后端在 SelectionDAG 层也没有能够匹配 vadd+vlshr 组合并生成 vavg 的模式规就。另外，SelectionDAG 阶段的多节点模式匹配复杂度更高，而且该阶段无法访问 LLVMIR 中的 nuw 等语义标记，因此不适合在此实现识别逻辑。

为此，本补丁在 IR 层实现了专属 PassLoongArchVectorIdiomPass。它在向量化结束后、SelectionDAG 指令选择开始之前扫描函数 IR，识别并替换以上惯用式，将其转化为对应的厂商 intrinsic 调用，保证后端指令选择器能够生成单条 vavg/vavgr 指令。

3.3.2 识别算法与 nuw 语义安全

Pass 以函数为单位运行，遍历所有基本块中的 lshr 指令作为候选入口，逐一调用核心识别函数 tryFoldAverageLshr。

识别算法的关键在于对 nuw 标记的严格检查。无符号平均运算(a+b)>>1 的语义正确性依赖

于加法不发生溢出。如果发生溢出，结果在无符号整数范围内会产生回绕，右移后就会得到错误的值。LLVMIR 中的 addnuw 标记明确断言该加法不存在无符号溢出，为 Pass 的替换提供了语义保证。如果 nuw 标记缺失，Pass 会直接跳过该指令，不执行任何改写，防止引入语义错误。

所识别的 IR 模式及其等价变体如下：

```
# 模式 A（截断平均）→vavg{Eu}
%sum = add nuw <N x iK> %a, %b
%r   = lshr <N x iK> %sum, splat(1)

# 模式 B（舍入平均）→vavgr{Eu}，支持三种等价写法
# 变体 B1：加法树形式
%sum1 = add nuw <N x iK> %a,    %b
%sum2 = add nuw <N x iK> %sum1, splat(1)
%r    = lshr <N x iK> %sum2, splat(1)

# 变体 B2：加 1 在内层
%a1   = add nuw <N x iK> %a, splat(1)
%sum  = add nuw <N x iK> %a1, %b
%r    = lshr <N x iK> %sum, splat(1)

# 变体 B3：加 1 在另一侧
%b1   = add nuw <N x iK> %b, splat(1)
%sum  = add nuw <N x iK> %a, %b1
%r    = lshr <N x iK> %sum, splat(1)
```

核心识别逻辑：

```
1.检查当前指令是 lshrV,splat(1)（右移 1 位）
2.检查向量类型属于 LSX128b 或 LASX256b 整数向量
3.检查 V 是 addnuwP,Q
4.判断是否为舍入平均（IsRounding）：
若 P 或 Q 为 splat(1)，对侧是 addnuwA,B→IsRounding=true
若 P 是 addnuwX,splat(1)，则 A=X,B=Q→IsRounding=true
若 Q 是 addnuwY,splat(1)，则 A=P,B=Y→IsRounding=true
否则 A=P,B=Q→IsRounding=false（截断平均）
5.根据(EltBits,IsLASX,IsRounding)查表选择 intrinsicID
6.用 IRBuilder 插入 intrinsic 调用，替换并删除原 lshr 链
```

替换完成后，函数的控制流图结构保持不变，仅将若干条算术指令替换为一条等价的 intrinsic 调用。因此，PreservedAnalyses 中保留 CFGAnalyses，无需重新运行控制流相关分析。

3.3.3 支持的向量类型与 Intrinsic 映射

Pass 覆盖 LSX（128b）和 LASX（256b）两种向量宽度，元素支持 i8/i16/i32/i64，共 16 种组合。

如表 3- 3 ，16 种组合与 LoongArchISA 手册中的 vavg.*/xvavg.*指令族一一对应。根据 UnofficialLoongArchIntrinsicsGuide 中 LA664 的硬件规格，每条 vavg/vavgr 指令的延迟与吞吐都是 2 周期，显著优于 vadd+vsrl 两条指令组合的 4 周期总延迟，且仅占用一个执行单元槽。

表 3- 3 支持的向量类型表

向量宽度	元素宽度	截断平均	舍入平均
LSX 128b	i8	loongarch.lsx.vavg.bu	loongarch.lsx.vavgr.bu
LSX 128b	i16	loongarch.lsx.vavg.hu	loongarch.lsx.vavgr.hu
LSX 128b	i32	loongarch.lsx.vavg.wu	loongarch.lsx.vavgr.wu
LSX 128b	i64	loongarch.lsx.vavg.du	loongarch.lsx.vavgr.du
LASX 256b	i8	loongarch.lasx.xvavg.bu	loongarch.lasx.xvavgr.bu
LASX 256b	i16	loongarch.lasx.xvavg.hu	loongarch.lasx.xvavgr.hu
LASX 256b	i32	loongarch.lasx.xvavg.wu	loongarch.lasx.xvavgr.wu
LASX 256b	i64	loongarch.lasx.xvavg.du	loongarch.lasx.xvavgr.du

3.3.4 正确性保证与可关闭性

语义安全：识别条件严格要求全部 add 带 nuw 标记，确保无符号语义正确；缺少 nuw 时 Pass 直接跳过，不引入任何改动。负测试（LIT 中带;CHECK-NOT:vavg 的用例）验证了 nuw 缺失时不触发替换。

统计可观测性：Pass 内置 NumAvgU 和 NumAvgrU 两个 LLVM_STATISTIC 计数器，分别统计截断平均与舍入平均的替换次数。在编译时附加-stats 标志，可精确量化该 Pass 对特定编译单元的贡献。

可关闭性：Pass 提供命令行开关--loongarch-enable-vector-idiom，默认为开启状态。将其设为 false 可在消融实验中完整关闭该 Pass，因此单独量化其对 TSVC2 性能的贡献。

回归测试覆盖：llvm/test/CodeGen/LoongArch/loongarch-vector-idiom.ll 涵盖全部 16 种类型路径，各路径都有 CHECK 指令验证目标汇编输出，以及 nuw 标记缺失时不触发替换的负向测试、LSX 与 LASX 同时存在的混合场景，以及控制流图保留性验证。

3.4 TTI 代价模型补全

3.4.1 TTI 在 LLVM 向量化流程中的作用

TTI（TargetTransformInfo）是 LLVM 中端向量化器与目标机器之间的"代价桥梁"。

LoopVectorizer 和 SLPVectorizer 在决定是否向量化、选择何种向量因子（VF）或混洗方式时，全部通过 TTI 接口查询指令代价，不依赖任何目标相关的硬编码逻辑。

若 TTI 未正确实现，向量化器只能使用基类的通用估算，这些估算通常是保守的"最坏情况"代价，不反映 LoongArch 的真实微架构特性。然后果主要有两类：

1. 向量化器误放弃：代价高估→向量化器认为"向量化收益<代价"→放弃向量化，实际应该向量化的循环用标量执行；

2. 向量化器误触发：代价低估→向量化器选择不合适的路径（如有序归约），实际生成低效代码。

本补丁共实现 9 类 TTI 接口（见下各节），系统地修正了以上两类问题，使向量化器能做出与 LA664 微架构吻合的决策。

3.4.2 getArithmeticInstrCost

问题：基类的通用估算对向量算术运算偏保守，对 LSX/LASX 不支持的运算不加区分，导致向量化器对基础算术运算高估代价而放弃向量化。

LoongArch 硬件特性：LSX/LASX 对整数与浮点的加减乘及位逻辑运算均提供专用的单周期吞吐指令，涵盖 vadd.*、vsub.*、vmul.*、vand.v 等，在 LA664 流水线上吞吐都是 1 周期每指令。

实现：对属于 FixedVectorType 且宽度≤寄存器宽度（LSX=128b，LASX=256b）的向量运算，以下操作码统一返回代价 1：

```
//整数与浮点基础算术：单周期吞吐→cost=1
case Instruction::Add: case Instruction::Sub: case Instruction::Mul:
case Instruction::And: case Instruction::Or:    case Instruction::Xor:
case Instruction::FAdd: case Instruction::FSub: case Instruction::FMul:
  return 1;
```

对于宽度超过单个寄存器的向量类型，以<16 x float>在 LSX 上需要两个 128 位寄存器为例，代价按切分后的寄存器数 NumVecInsts 线性累积，以准确反映多寄存器运算的实际开销。

对于 LoongArch 不支持的向量操作，如整数向量除法，本接口不处理，回退至基类实现，因此触发极高的标量化代价路径，引导向量化器放弃该方向。

3.4.3 getMemoryOpCost

问题：基类不区分对齐与未对齐访存，统一给出相同代价。在 LA664 上，LSX/LASX 向量 load/store 要求 16 字节对齐（对 LSX128b）；若地址未对齐，硬件会执行两次访问并在内部合并，实际延迟接近两倍。若向量化器低估未对齐代价，会错误地选择向量化未对齐访问，运行时引起性能下降。

实现：基础代价取 NumVecInsts，即操作所需的向量寄存器数量。在这个基础上，根据对齐

信息施加惩罚：

```
InstructionCost Cost = NumVecInsts;
const uint64_t PreferredAlign = std::min<uint64_t>(16, StoreSize);
if (PreferredAlign > 0 && Alignment < Align(PreferredAlign))
   Cost += NumVecInsts;     // 未对齐：代价翻倍
return Cost;
```

对齐判断依据编译期已知的 Alignment 信息，该信息来源于指针的 align 属性或 alignof 静态推断。若编译期无法确定对齐情况，就采取保守策略，不施加额外惩罚，由运行时对齐检测路径进行处理。

3.4.4 getShuffleCost

表 3- 4 代价估算表

ShuffleKind	LoongArch 目标指令	代价估算
广播单元素	vreplvei.* / xvreplvei.*	有 Mask：NumVecInsts； 无 Mask：2×NumVecInsts
全倒序	vshuf4i 配置	NumVecInsts
按位选择	vbitsel.v	NumVecInsts
简单交错掩码	vpackev / vpackod	NumVecInsts
任意掩码	通用多指令序列	2×NumVecInsts
SK_Transpose	LASX 原生支持；LSX 需两步	LASX：NumVecInsts；LSX： 2×NumVecInsts

基类对所有混洗操作统一返回固定代价 1，未区分 pack、unpack、broadcast、transpose 等操作的实际差异。错误的代价估算可能导致 SLP 向量化器选择代价高昂的通用混洗路径，或反之放弃本可高效实现的交错向量化。

isSimpleInterleaveMask 为本补丁新增的辅助函数，用于判断掩码是否属于标准交错模式：偶索引选择模式[0,N,2,N+2,...]对应 vpackev 指令，奇索引选择模式[1,N+1,3,N+3,...]对应 vpackod 指令，都可以以单条指令高效实现。

3.4.5 getCastInstrCost

表 3- 5 转换操作代价估算表

ShuffleKind	LoongArch 目标指令	代价估算
SK_Broadcast	vreplvei.*/xvreplvei.*	有 Mask：NumVecInsts 无 Mask：2×NumVecInsts
SK_Reverse	vshuf4i 配置	NumVecInsts
SK_Select	vbitsel.v	NumVecInsts
SK_PermuteTwoSrc	vpackev/vpackod	NumVecInsts
SK_PermuteTwoSrc	通用多指令序列	2×NumVecInsts
SK_Transpose	LASX 原生支持，LSX 需两步	LASX：NumVecInsts； LSX：2×NumVecInsts

问题：基类将 BitCast 和真实类型转换一视同仁地估算代价。在 LoongArch 上，BitCast（如 <4xfloat>→<2xdouble>）仅是寄存器解释方式变化，无需任何指令；若被估算为非零代价，SLP 会因"bitcast 有代价"而放弃构造混合类型向量，损失向量化机会。

涉及 i1 的转换需要先做比较生成掩码再做位选，需三条指令，代价 3×反映了这一开销，防止向量化器生成大量 i1 向量比较并期望它们"廉价"执行。

3.4.6 getArithmeticReductionCost

问题：归约代价是 LoongArchTTI 中被误估最严重的接口。基类往往严重低估有序 FP 归约的代价，导致 LoopVectorizer 对 for(sum+=a[i])类循环选择"有序归约向量化"——循环体向量化求和，但累加器顺序标量相加，实际产生 N 次 vpickve+fadd 的抽取链，反而比纯标量更慢。

实现：针对不同归约路径分别精确建模：

有序 FP 归约：

代价设定为 NumElements×2，以每个元素需要一次向量抽取（延迟 2 周期）和一次标量浮点加法（延迟 1 周期）为依据建模。该代价通常高于纯标量归约路径，使向量化器放弃有序归约，转而选择标量路径或允许运算重排的无序向量归约路径。

无序 FP 归约：

此时可利用 VBSRL 树结构将向量逐步折叠至标量：

```
LSX 128b v4f32 FADD VBSRL 树（Steps = log2(4) = 2）：
  代价 = 2×Steps + 1 = 2×2 + 1 = 5
  （2 次 vbsrl + 2 次 vfadd.s + 1 次元素 0 免费抽取）

LASX 256b v8f32 FADD VBSRL 树：
  代价 = 2×ChunkSteps + 3 = 2×2 + 3 = 7
  （先上半折叠至下半：1 次 xvpermi + 1 次 vfadd.s，再 LSX 路径：5 次，共 7 次）
```

整数加法归约：采用向量水平宽化加法 VHADDW 链式归约，每步将向量宽度折半，代价为 (Steps+1)×NumVecInsts。

位逻辑归约（And/Or/Xor）：采用 VBSRL 结合对应向量位运算指令构成树形归约，代价为 (2×Steps+1)×NumVecInsts。

以上精确代价让向量化器能够准确评估各归约路径的实际开销，在点积、求和及最大值等常见 TSVC2 测试 kernel 中做出最优决策，是 vdotr、s313、s353 等 kernel 取得显著加速的基础。

3.4.7 getMinMaxReductionCost

实现：本接口对应后端 lowerVECREDUCE 中各类 Min/Max 归约操作的 VBSRL 树实现，代价建模方法与 getArithmeticReductionCost 中的无序浮点归约路径保持一致：

LSX128bFP（v4f32/v2f64）：2×Steps+1

LASX256bFP（v8f32/v4f64）：2×ChunkSteps+3

LASX256b 整数：由于不存在原生的 256 位整数 Min/Max 归约指令，采用 Expand 路径，将其拆分为两个独立的 128 位归约后再标量合并，代价按此路径相应计算

整数 Min/Max（LSX128b）：2×Steps+1

Min/Max 归约代价与算术归约保持一致，使向量化器对包含向量最大值归约的循环同样能做出正确的向量化决策。

3.4.8 getIntrinsicInstrCost

基类对 llvm.fma、llvm.sqrt、llvm.abs 等常用 intrinsic 采用与目标架构无关的固定代价估算。在 LoongArch 上，FMA 指令的实际吞吐显著优于保守估算值，而 sqrt 因非完全流水线设计，其延迟远高于基础算术运算。将 FMA 的代价从基类默认的 2×修正为 1×，向量化器在识别到 FMA 融合机会时将更倾向于选择融合路径，对 TSVC2 中大量乘加型 kernel 具有直接的性能提升效果。

基于 UnofficialLoongArchIntrinsicsGuide 的 LA664 指令表，对以下 intrinsic 提供精确的单周期吞吐代价。

表 3- 5 单周期吞吐代价表

Intrinsic（向量/标量均适用）	LA664 指令	代价
llvm.fma、llvm.fmuladd	vfmadd.s/d	1×
llvm.fabs、llvm.copysign、llvm.canonicalize	向量位运算实现	1×
llvm.sqrt	vfsqrt.s/d	8×
llvm.smax、llvm.smin、llvm.umax、llvm.umin	vmax.*/vmin.*	1×
llvm.maxnum、llvm.minnum、llvm.maximum、llvm.minimum	vfmax.*/vfmin.*	1×
llvm.abs、llvm.ctpop	vabsd.*/vpcnt.*	1×
llvm.ctlz、llvm.cttz	vclz.*/vctz.*	1×
llvm.bswap、llvm.bitreverse	需要混洗辅助	2×

3.4.9 getMemIntrinsicInstrCost 与掩码访存显式拒绝

LoongArchLSX/LASX 硬件没有原生掩码加载/存储指令。若不显式声明不合法，向量化器在处理含条件写的循环时，在循环尾部折叠场景中会尝试生成 masked_store，最终到达后端时无法合法化，导致编译器报错或产生极慢的软件仿真代码。

实现：在 TTI 头文件中显式声明所有掩码访存接口都不合法，同时为 masked_load/masked_store/vp_load/vp_store/masked_gather/masked_scatter 返回极高代价，使向量化器的代价分析在选择这条路径时得到一个明确的"禁止"信号，不会因为"代价虽高但仍选"而走错路。

这一双重保障确保向量化器在循环尾部不会生成掩码访存，而是选择剥离或不向量化，保证生成代码的合法性和性能。

3.4.10 全局向量化策略配置

除以上代价接口外，TTI 还提供若干布尔型配置接口，直接控制向量化器的全局策略：

getMinVectorRegisterBitWidth：返回 128

LoopVectorizer 在探索向量因子（VF）候选集时，从 getMinVectorRegisterBitWidth 开始，以 2 的幂次向上枚举到 getRegisterBitWidth。若 getMinVectorRegisterBitWidth 返回 256，所有 VF=4 的候选会被跳过，导致在寄存器压力较大在循环体变量较多的场景下只有 VF=8 可选，可能因寄存器溢出而引起回归。固定返回 128 使 VF=4 和 VF=8 均进入候选集，由收益分析自主决定最优 VF。

enableInterleavedAccessVectorization：在系统支持 LSX 时返回 true，以激活 InterleavedAccessPass 的 TTI 查询入口。若该接口返回 false，就 Pass 即便完成注册也不会执行任何改写，因为 Pass 内部将首先查询此接口以确认目标支持。

enableOrderedReductions：有 LSX 时返回 true，允许 LoopVectorizer 对浮点归约循环选择"有序归约向量化"路径——循环体向量化并行乘/加，但累加器仍按顺序标量 fadd，保持严格的 IEEE-754 浮点顺序语义，适合需要结果可重现的科学计算场景。关闭时只有 reassoc 标记的循环才能用向量归约。

shouldMaximizeVectorBandwidth：返回 false，不强制最大化向量带宽。若该接口返回 true，LoopVectorizer 将倾向于选取最大向量因子，而不考虑循环体的寄存器压力。对于 LoongArch 具备 32 个通用向量寄存器的架构而言，在寄存器密集型循环中盲目最大化向量因子将导致寄存器溢出，反而引入性能损失。

3.5 后端 Lowering 与代码生成

后端 Lowering 处于编译流水线的末端，直接决定最终机器码的质量。本节各项优化都基于 SelectionDAG 的 CustomLowering 机制实现：在 LoongArchTargetLowering 的构造函数中，将特定 ISD 节点声明为 Custom 操作，再在 LowerOperation 方法中把它分发到对应的专属处理函数。

3.5.1 浮点向量归约 CustomLowering

问题：LSX/LASX 浮点向量的 VECREDUCE_FADD、VECREDUCE_FMUL、VECREDUCE_FMAX、VECREDUCE_FMIN、VECREDUCE_FMAXIMUM、VECREDUCE_FMINIMUM 在 LLVM22.1.3 基线中都走 Expand 路径：SelectionDAG 将其展开为从 lane[1]开始逐元素抽取（EXTRACT_VECTOR_ELT）再标量运算的链式序列。N 个元素产生 N-1 次抽取加 N-1 次标量浮点运算，每步都有数据依赖，无法乱序，是明显的瓶颈。

例如对 v4f32 的 VECREDUCE_FADD，基线展开后的汇编为：

```
;基线 Expand 路径：4 条串行指令，全部数据依赖链
vpickve.w   $f1, $vr0, 1        ; 抽取 lane[1]（2 cycle 延迟）
fadd.s      $f0, $f0, $f1       ; 依赖上一条（1 cycle）
vpickve.w   $f2, $vr0, 2        ; 抽取 lane[2]（可与 fadd.s 乱序）
fadd.s      $f0, $f0, $f2       ; 依赖前一条 fadd.s
vpickve.w   $f3, $vr0, 3        ; 抽取 lane[3]（同上）
fadd.s      $f0, $f0, $f3       ; 最终结果
```

实际关键路径长度：(2+1)×3=9 周期（3 对抽取+加法串行）。

实现：将以上 6 种浮点归约节点在 LSX（v4f32/v2f64）和 LASX（v8f32/v4f64）上全部声明为 Custom：

```
//构造函数中（LSX 类型，LASX 同理）
for (MVT VT : {MVT::v4f32, MVT::v2f64}) {
  setOperationAction(ISD::VECREDUCE_FADD, VT, Custom);
  setOperationAction(ISD::VECREDUCE_FMUL, VT, Custom);
  setOperationAction(ISD::VECREDUCE_FMAX, VT, Custom);
  setOperationAction(ISD::VECREDUCE_FMIN, VT, Custom);
  setOperationAction(ISD::VECREDUCE_FMAXIMUM, VT, Custom);
  setOperationAction(ISD::VECREDUCE_FMINIMUM, VT, Custom);
}
```

LSX128bVBSRL 树（以 v4f32FADD 为例）：

利用 VBSRL.V（向量字节右移逻辑）将高半部分"移"到低半，再与低半做向量加，循环对折直到结果集中在 lane：

```
; Custom VBSRL 树路径：5 条指令，关键路径 ≈ 5 周期
vbsrl.v     $vr1, $vr0, 8       ; 将 [2,3,_,_] 移到 [0,1] 位置
vfadd.s     $vr0, $vr0, $vr1 ; lane[0]+=lane[2], lane[1]+=lane[3]（向量并行）
vbsrl.v     $vr1, $vr0, 4       ; 将 lane[1] 移到 lane[0] 位置
vfadd.s     $vr0, $vr0, $vr1 ; lane[0] = 最终结果
; lane[0] 免费抽取（见 3.4.4），无需额外指令
```

键路径长度为 4 周期，远优于 Expand 路径的 9 周期。此外，第一轮 vbsrl.v 与第一轮 vfadd.s 的输入之间不存在数据依赖，在超标量处理器上可并发执行，实际执行窗口进一步缩短。

v2f64 路径：只需要一次 vbsrl.v 移位与一次 vfadd.d，共 2 条指令，相较于基线路径的 2 次抽取加 1 次标量加法共 3 条指令，指令数减少约三分之一。

LASX256b 路径（以 v8f32FADD 为例）：

LASX256 位向量由高 128 位（lane[4..7]）和低 128 位（lane[0..3]）两部分构成。基线 Expand 路径将其拆分为两条独立的 128 位归约链，最后通过标量运算合并，共需 11 条指令。Custom 路径先通过单条 XVPERMI.Q 将高半 128 位折叠至低半，再对所得 128 位向量执行标准的 VBSRL 树归约：

```
step1: xvpermi.q $xvr1, $xvr0, 0x01   ; 提取上半 128b 到 xvr1[0..127]
step2: vfadd.s    $vr0, $vr0, $vr1     ; 上半+下半 → 4 个元素结果在 vr0
step3..5: 标准 LSX VBSRL 树（同上）
```

共计 6 条指令，较基线减少 5 条，且消除了两条独立归约链之间标量合并的串行瓶颈。

其他归约类型的 Custom 路径结构与 FADD 完全一致，仅将对应的向量运算指令替换为相应的变体。

3.5.2 交错读 Custom Lowering

科学计算与多媒体处理领域广泛采用结构数组布局，以复数数组为例，其内存排列为[re0,im0,re1,im1,re2,im2,...]。向量化后，循环体每次加载 2N 个连续元素并进行解交错操作，在 LLVMIR 层面表达为：

```
%wide = load <8 x float>, ptr %p          ; 一次宽加载 8 个 float
%real = shufflevector %wide, poison, [0, 2, 4, 6]   ; 取偶索引（实部）
%imag = shufflevector %wide, poison, [1, 3, 5, 7]   ; 取奇索引（虚部）
```

基线后端采用通用 shufflelowering 处理此模式，通常产生多条逐元素抽取与插入指令，代价极高。InterleavedAccessPass 负责识别并改写该模式，但改写后的 IR 仍需由 lowerInterleavedLoad 在后端进一步处理，才能生成 LoongArch 专属的高效指令序列。

实现：在 lowerInterleavedLoad 中处理 Factor=2 和 Factor=4 两种情况：

Factor=2 路径：

```
改写策略：
  1. 将宽 load 拆成两个独立的窄 load（各加载 N/2 个向量寄存器宽度）
  2. 用 IR shufflevector 生成 even/odd mask 的分量：
       even mask [0,2,4,...] → vpackev（打包偶索引元素）
       odd  mask [1,3,5,...] → vpackod（打包奇索引元素）

生成汇编示例（8 个 float，v4f32×2）：
  vld     $vr0, %p, 0          ; 加载低 4 float
  vld     $vr1, %p, 16         ; 加载高 4 float
```

```
vpackev.w $vr_re, $vr0, $vr1    ; 打包偶索引 → 实部 [re0,re1,re2,re3]
vpackod.w $vr_im, $vr0, $vr1    ; 打包奇索引 → 虚部 [im0,im1,im2,im3]
```

相比基线逐元素操作需要 16 条指令，优化后只需要 4 条指令，理论吞吐提升约 4 倍。

Factor=4 路径（RGBA 像素、四分量结构）：

需要两层 pack/unpack 树，完全分离 4 个通道：

```
输入：A=[r0,g0,b0,a0, r1,g1,b1,a1] 和 B=[r2,g2,b2,a2, r3,g3,b3,a3]

Layer 1（分离奇偶对）：
  P0 = vpackev(A, B) = [r0,b0,r2,b2]    (取 A 和 B 的偶索引)
  P1 = vpackod(A, B) = [g0,a0,g2,a2]    (取 A 和 B 的奇索引)
  P2 = vpackev(C, D) = [r1,b1,r3,b3]    (下一对同理)
  P3 = vpackod(C, D) = [g1,a1,g3,a3]

Layer 2（进一步分离）：
  Lane_R = vpackev(P0, P2) = [r0,r1,r2,r3]    ; R 通道
  Lane_G = vpackev(P1, P3) = [g0,g1,g2,g3]    ; G 通道
  Lane_B = vpackod(P0, P2) = [b0,b1,b2,b3]    ; B 通道
  Lane_A = vpackod(P1, P3) = [a0,a1,a2,a3]    ; A 通道
```

共生成 8 条 vpackev/vpackod 指令，对于不同元素宽度可替换为对应的 vilvl/vilvh 变体，相较基线所需 16 次逐元素操作大幅降低指令数。

交错写返回 false，保留通用路径。原因是 LoongArch 后端在 VECTOR_SHUFFLElowering 阶段已经能识别标准的交错 interleave 掩码（[0,N,1,N+1,...]）并选出 VILVL/VILVH 对，IR 层无需额外改写。而对 store 做 IR 层重写存在与这一现有逻辑冲突的回归风险，经测试在 TSVC2 上 store 路径的现有代码质量已经较好，故不引入改写。

getMaxSupportedInterleaveFactor 返回 4，向 InterleavedAccessPass 声明目标最大可处理 Factor=4 的交错模式，覆盖 RGBA 四通道等典型场景。Factor 大于 4 时不存在对应的高效指令序列，Pass 对此情况退回通用处理路径。

3.5.3 激进 FMA 融合

问题：LA664 上的向量 FMA 指令具备高效的吞吐特性，延迟 5 周期、吞吐 2 周期，与独立执行 fmul+fadd 相近，但合并为单条指令可减少执行槽占用并缩短寄存器的活跃区间。

DAGCombiner 默认只在 fmul 结果仅有一个 use 时才融合 FMA，以避免将 fmul 的结果值"消费掉"后其他 use 无法复用。然而，enableAggressiveFMAFusion 开启后，DAGCombiner 会在 fmul 的结果有多个 use 时，选择复制一份专用于 FMA 融合的 fmul，将 fmul+fadd 对合并为 FMA 并保

留其他 use 的 fmul，在有多个乘加运算共享乘法结果的场景生成更紧凑的代码。

激进 FMA 融合只在 IR 携带 contract 或 reassocfast-math 标记时由 DAGCombiner 实际执行。在缺少以上标记的情况下，IEEE-754 标准要求乘法结果先行舍入再进行加法，融合为单条 FMA 将改变舍入顺序，违反浮点标准，因此 DAGCombiner 不会进行融合。本项改动不破坏任何需要严格浮点语义的代码。

3.5.4 元素 0 免费抽取

原理：LoongArch 指令集架构规定向量寄存器与标量浮点寄存器共享同一物理寄存器文件。具体而言，128 位向量寄存器的低 32 位与标量单精度浮点寄存器完全物理重叠，其低 64 位就与双精度浮点寄存器对应同一寄存器文件条目的不同数据视图。

因此，从向量寄存器中提取第 0 号元素无需任何实际指令，只需要在寄存器分配阶段将该向量寄存器对应的子寄存器作为浮点寄存器使用即可。LLVM 通过 EXTRACT_SUBREG 伪操作表达此寄存器别名关系，该伪操作不生成任何机器码。

实现：在 LoongArchLSXInstrInfo.td 和 LoongArchLASXInstrInfo.td 中添加 TableGenPat 规就，覆盖 f32 和 f64 元素。

在 VBSRL 树归约中，最终结果被折叠到 lane，此时本优化发挥关键作用——使归约链末尾的 VPICKVE2GR 被完全消除，整个归约链的指令数再减 1。

修改前：

```
vbsrl.v      $vr1, $vr0, 8
vfadd.s      $vr0, $vr0, $vr1
vbsrl.v      $vr1, $vr0, 4
vfadd.s      $vr0, $vr0, $vr1
vpickve.w    $f0,   $vr0, 0        ; ← 多出来的 1 条
; ret 用 $f0 / $fa0
```

修改后：

```
vbsrl.v      $vr1, $vr0, 8
vfadd.s      $vr0, $vr0, $vr1
vbsrl.v      $vr1, $vr0, 4
vfadd.s      $vr0, $vr0, $vr1
; 无 vpickve：ABI 返回值寄存器与 $vr0.lane0 别名即可
; EXTRACT_SUBREG 在编译期消掉，目标文件里看不到抽取指令
```

3.5.5 整数 trunc 的 pack-shuffle

x264 一类循环向量化之后，经常要把 8 个 i32 截成 8 个 i8 再写回像素。基线只在 128 位源上会

走 pack；256 位 LASX 走不通，合法化阶段就会对每个车道做 xvpickve2gr，再 xvpermi.q 拼回去，指令又多又慢。

方法：在 LoongArchISelLowering 里实现 lowerVectorTruncateToPackShuffle。把整数向量 trunc 看成每隔 N 字节取一个元素，用 bitcast 加上 vpackev/vpicke 一类 pack shuffle 收窄，128 位和 256 位都覆盖。

源码（向量化后）：

```
uint8_t dst[8];

int32_t src[8];

for (int i = 0; i < 8; i++)

   dst[i] = (uint8_t)src[i];     // trunc i32 → i8
```

对应 IR：

```
%t = trunc <8 x i32> %src to <8 x i8>

store <8 x i8> %t, ptr %dst
```

改前：

```
xvpickve2gr.w   $t0, $xr0, 0

andi                $t0, $t0, 0xff

xvpickve2gr.w   $t1, $xr0, 1

andi                $t1, $t1, 0xff

; …共 8 次，再用 xvpermi.q / 插入拼成字节向量
```

改后：

```
xvpickev.h   $xr1, $xr0, $xr0    ; 每个 i32 留下低 16 位 → 8×i16

xvpickev.b   $xr1, $xr1, $xr1    ; 每个 i16 留下低 8 位  → 8×i8

; 无逐道 xvpickve2gr
```

3.5.6 vext2xv 与整数 SAD 归约

整数 SIMD 还有两端：变宽和水平求和。基线两端都容易抽到通用寄存器。

x264 运动补偿加权一类，会把 8 个 i8 扩成 8 个 i32 再做乘加。基线对每个字节 vpickve2gr 再插回去。

方法：lowerExtendVectorInReg 识别寄存器内的零扩展/符号扩展，降成 vext2xv.*。128 位没有对应指令时，用 vilvl 对零做 unpack。

源码：

```
int32_t w[8];

uint8_t p[8];

for (int i = 0; i < 8; i++)

    w[i] = (int32_t)p[i];          // zext i8 → i32
```

对应 IR：

```
%w = zext <8 x i8> %p to <8 x i32>
```

改前：8 次抽字节、扩宽、再插回向量。

改后：

```
vext2xv.wu.b    $xr1, $vr0        ; 低 8 个 i8 一次变成 8 个 i32
```

SAD 一类是先扩成 i32，再把一向量加总成一个标量。基线同样容易拆成标量加。

方法：整数 VECREDUCE_ADD 走 lowerVECREDUCE_ADD，反复 vhaddw，最后取 lane0。256 位 LASX 再用 xvpermi 把上半折下来相加。LLVM 里非法的 512 位 v16i32（宽 SAD）先拆成两条 256 位，各做完再标量加。

源码：

```
int32_t s = 0;

for (int i = 0; i < 8; i++)

    s += absdiff[i];                  // 向量化后是 vecreduce.add
```

对应 IR：

```
%sum = call i32 @llvm.vector.reduce.add.v8i32(<8 x i32> %absdiff)
```

改前：逐道 xvpickve2gr+标量 add。

改后：

```
vhaddw.d.w       $vr0, $vr0, $vr0

vhaddw.q.d       $vr0, $vr0, $vr0

vpickve2gr.d    $a0, $vr0, 0        ; 只抽最终和

v16i32 则先拆成两个 v8i32，各自走上面的链，再 $a0 += $a1。
```

3.6 瓶颈问题与解决方法

本章记录补丁开发过程中遇到的主要工程瓶颈，按问题类型分节介绍，每节依次说明问题表现、根本原因、解决方案和验证方式。所有问题都已在提交的补丁中得到解决，相关 LIT 测试作为回归防护持续覆盖。

3.6.1 向量化未触发

问题表现：

在基线编译器中，对 v4f32 类型的简单加法循环启用-Rpass-missed=loop-vectorize 后，编译器输出如下诊断信息：

```
remark: loop not vectorized: instruction cost too high (cost=16, threshold=12)
```

类似现象广泛存在于算术密集型 kernel 中，导致大量本应受益于向量化的循环未能生成 SIMD 代码。

原因分析：

LLVM 的 LoopVectorizer 在评估是否向量化时，通过 TTI 的 getArithmeticInstrCost 接口查询循环体内每类算术运算的执行代价。在官方基线的 LoongArchTTIImpl 中，该接口未被覆写，调用链回退至 BasicTTIImpl::getArithmeticInstrCost。基类实现以"将向量运算展开为标量序列"的悲观模型估算代价，对<4xfloat>加法返回 4（等同于 4 条标量加法），而 LoongArchLSX 的 vfadd.s 实际上以单指令、单周期吞吐完成 4 个浮点元素的并行加法。代价估算值 4 与实际代价 1 之间的差距使向量化收益计算产生系统性偏差：LoopVectorizer 认为向量化版本的总指令代价高于标量版本，因此错误地放弃向量化。

解决方案：

在 LoongArchTTIImpl::getArithmeticInstrCost 中，对 LSX/LASX 向量类型的整数与浮点算术运算统一返回代价 1，精确反映 LoongArchSIMD 指令的单周期吞吐特性，对于无对应向量指令的操作（如整数除法），保留回退至基类的行为，避免过度优化引发功能错误。

验证方式：

通过 opt-passes="print<cost-model>"-mtriple=loongarch64-mattr=+lsx 确认 v4f32 加法的代价输出从 4 下降至 1；通过-Rpass=loop-vectorize 确认原先报告 costtoohigh 的 kernel 在补丁后输出 vectorizedloop。配套 LIT 位于 test/Analysis/CostModel/LoongArch/，对各算术指令的代价值进行 CHECK 回归。

3.6.2 FP 向量归约退化

问题表现：

对 TSVC2 中 vdotr 等包含浮点向量归约的 kernel，在基线编译器生成的汇编中观察到如下模式：

```
vpickve2gr.w    $t0, $vr0, 1
movgr2fr.w       $f1, $t0
fadd.s              $f0, $f0, $f1
vpickve2gr.w    $t0, $vr0, 2
movgr2fr.w       $f2, $t0
fadd.s              $f0, $f0, $f2
; ...（共 N-1 次重复）
```

该序列中每条 fadd.s 依赖前一条的结果，形成不可并行的串行依赖链，执行延迟与元素个数 N 线性相关，严重限制了流水线利用率。

根本原因分析:

LLVM SelectionDAG 在处理 VECREDUCE_FADD 等浮点归约节点时，优先查询目标后端是否注册了 Custom 处理。若未注册，就走 Expand 路径，将归约分解为逐元素的抽取与标量累加序列。在官方基线的 LoongArchTargetLowering 中，VECREDUCE_FADD、VECREDUCE_FMUL 等 6 类浮点归约节点均未调用 setOperationAction(...,Custom)，SelectionDAG 因此自动选择 Expand 路径，生成以上低效的串行抽取链。

解决方案:

在 LoongArchTargetLowering 构造函数中，对 LSX 与 LASX 向量类型分别注册 6 类浮点归约节点为 Custom：

```
for (MVT VT : {MVT::v4f32, MVT::v2f64}) {
  setOperationAction(ISD::VECREDUCE_FADD, VT, Custom);
  setOperationAction(ISD::VECREDUCE_FMUL, VT, Custom);
  // FMAX/FMIN/FMAXIMUM/FMINIMUM 同上
}
```

在 lowerVECREDUCE 中实现 VBSRL 树。对 N 个元素的向量，将高半部分移至低半，再与低半执行向量二目运算，递归直至向量宽度缩减为一个元素宽度：

```
for (int i = NumEles; i > 1; i /= 2) {
  SDValue ShiftAmt = DAG.getConstant(i * EleBits / 16, DL, GRLenVT);
  SDValue Tmp = DAG.getNode(LoongArchISD::VBSRL, DL, VecTy, Val, ShiftAmt);
  Val = DAG.getNode(Opcode, DL, VecTy, Val, Tmp);
}
```

归约结果位于向量 lane，通过 EXTRACT_VECTOR_ELT 的零代价 EXTRACT_SUBREGPat 提取。

验证方式：

通过 llc-mtriple=loongarch64-mattr=+lsx 检查归约 kernel 的汇编输出，确认 vbsrl.v 指令出现且 vpickve2gr.w 抽取循环消失。配套 LIT 位于 test/CodeGen/LoongArch/lsx/vec-reduce-fp.ll，逐行 CHECK 生成的向量归约指令序列。

3.6.3 交错访存优化未生效

问题表现：

对 TSVC2 中 s118 等以步长 2 访问数组元素的 kernel，基线编译器生成的汇编中出现逐元素标量 load 而非向量打包指令，访存带宽利用率极低。预期应生成 vpackev.w/vpackod.w 等向量打包指令以充分利用 LSX 访存带宽。

根本原因分析：

LLVM 的交错访存优化依赖两个相互独立的组件协同工作。中间层的 InterleavedAccessPass 负责识别 IR 中满足交错模式的 shufflevector 指令组合，后端的 lowerInterleavedLoad 负责将识别到的交错模式 IR 改写为目标特化的窄 load 加 pack 指令序列。

InterleavedAccessPass 在运行前会查询 TTI 的 enableInterleavedAccessVectorization 接口，若返回 false，就 Pass 直接跳过当前目标，不对任何交错模式进行处理。官方基线的 LoongArchTTIImpl 未覆写该接口，基类默认返回 false，导致 InterleavedAccessPass 完全不处理 LoongArch 目标。即使在 addIRPasses 中注册了该 Pass，若后端未实现 lowerInterleavedLoad，IR 层的识别结果也无法转化为有效的代码改写。官方基线两端均缺失，交错访存优化完全无法生效。

解决方案：

需同时完成三项工作以实现端到端的交错访存优化：

第一步，在 LoongArchTTIImpl 中覆写 enableInterleavedAccessVectorization，对开启 LSX 的目标返回 true，通知 InterleavedAccessPass 当前目标支持交错优化。

第二步，在 LoongArchTargetMachine::addIRPasses 中注册 InterleavedAccessPass，确保在优化级别不低于 -O1 时该 Pass 进入 IR 优化流水线。

第三步，在 LoongArchTargetLowering 中实现 lowerInterleavedLoad，对 Factor=2 路径发出两个连续窄 load，分别加载偶数与奇数地址块，再以 vpackev/vpackod 实现解交错；对 Factor=4 路径发出 4 个窄 load，以 vpackev/vpackod/vilvl/vilvh 等打包指令的组合完成 4 路解交错。

验证方式：

通过 -Rpass=loop-vectorize 与 llvm-objdump-d 确认 s118kernel 的汇编中出现 vpackev.w/vpackod.w 指令，且逐元素标量 load 消失。配套 LIT 位于 test/Transforms/InterleavedAccess/LoongArch/，通过 FileCheck 验证 Factor=2 与 Factor=4 路径的 IR 改写结果与预期一致。

修改前：

```
%wide = load <8 x float>, ptr %b, align 4
%even = shufflevector <8 x float> %wide, <8 x float> poison,
                      <4 x i32> <i32 0, i32 2, i32 4, i32 6>
%odd  = shufflevector <8 x float> %wide, <8 x float> poison,
                      <4 x i32> <i32 1, i32 3, i32 5, i32 7>
%sum  = fadd <4 x float> %even, %odd
```

修改后：

```
%chunk0 = load <4 x float>, ptr %b, align 4                ; 连续块 0
%chunk1 = load <4 x float>, ptr %b.ia, align 4             ; 连续块 1（+4 个 float）
%even   = shufflevector %chunk0, %chunk1, <0,2,4,6>   ; → vpackev.w
%odd    = shufflevector %chunk0, %chunk1, <1,3,5,7>   ; → vpackod.w
%sum    = fadd <4 x float> %even, %odd
```

3.6.4 FMA 指令未生成

问题表现：

对包含乘加运算的浮点计算 kernel（如 TSVC2 的 s314），基线编译器生成独立的 vfmul.s 与 vfadd.s 指令对，而非单条 vfmadd.s 融合乘加指令。在 LA664 上，vfmadd.s 的吞吐与 vfadd.s 相同，但将两条指令合并为一条可减少指令发射压力，并消除浮点乘法中间结果的舍入误差。

根本原因分析：

LLVM 的 DAGCombiner 在处理 fmul+fadd 模式时，通过查询 TargetLowering::enableAggressiveFMAFusion 决定是否对多次使用的乘法结果执行 FMA 融合。该接口有两个重载版本：基于 EVT 的版本用于 SelectionDAG 层，基于 LLT 的版本用于 GlobalISel 层。官方 LoongArch 基线两个版本均未覆写，基类默认返回 false，DAGCombiner 只对 fmul 结果仅被一处使用的情况执行融合。当 fmul 结果被多处使用时（典型场景为点积循环中的乘加累加），融合被跳过，产生独立的乘法与加法指令。

解决方案：

LoongArchTargetLowering 中同时覆写 EVT 与 LLT 两个版本的 enableAggressiveFMAFusion，对所有浮点标量与向量类型返回 true。 此设置对 f32/f64 标量路径及 v4f32/v2f64/v8f32/v4f64 向量路径全部生效，使 DAGCombiner 在浮点重排序允许的条件下尽可能生成 vfmadd.s/vfmadd.d 指令。

修改前：

```
vfmul.s     $vr1, $vr2, $vr3        ; 独立乘法
vfadd.s     $vr0, $vr0, $vr1        ; 独立加法
; 两条指令，多一次中间舍入
```

修改后：

```
vfmadd.s    $vr0, $vr2, $vr3, $vr0     ; 单条融合乘加
; 发射压力下降，中间结果不再单独舍入
```

3.6.5 掩码访存导致后端合法化失败

问题表现：

在开发 TTI 代价模型的早期阶段，部分含条件赋值的循环（如 s4121）在尾部折叠优化路径下产生 masked_load/masked_storeIR 节点。由于 LoongArch 后端未注册这类节点的合法化动作，SelectionDAG 合法化阶段无法为其选择指令，触发编译器内部断言失败，导致编译终止。

```
// 条件写 + 迭代次数未必整除向量宽度
void s4121_like(float *a, float *b, float *c, int n) {
   for (int i = 0; i < n; ++i) {
      if (c[i] > 0.0f)                // 条件执行
         a[i] = b[i] + 1.0f;          // 条件 store
   }
}
```

根本原因分析：

LLVM 的 LoopVectorizer 在向量化包含条件执行的循环时，对于尾部迭代次数不能整除向量宽度的情况，存在两种处理策略：其一为生成尾部标量循环，其二为使用掩码向量访存对齐向量边界。当 TTI 对 isLegalMaskedLoad 未明确返回 false 时，某些优化路径会绕过严格的合法性检查，错误地生成掩码访存 IR 节点。LoongArchLSX/LASX 不提供原生掩码向量访存指令，后端无法将 masked_load 合法化，进一步触发 llvm_unreachable。

解决方案：

本补丁采用双重防护机制，从 TTI 层根本上阻止掩码访存路径被选择。在 getMemIntrinsicInstrCost 中对所有掩码与向量谓词 intrinsic 返回高代价值，对 isLegalMaskedLoad/isLegalMaskedStore/isLegalMaskedGather/isLegalMaskedScatter 等接口显式返回 false，防止任何依赖这些接口的优化路径生成掩码 IR。

3.6.6 LASX256 位 FP 归约产生冗余双链

问题表现：

对 LASXv8f32 类型的浮点归约，在实现 7.2 节的 VBSRL 树之前，观察到编译器生成了两条独立的 128 位归约链，最终通过两次标量提取与一次标量加法完成合并，总指令数约为 11 条。这一结果优于 Expand 路径的逐元素抽取，但仍劣于理论最优的单条折叠路径。

根本原因分析：

LLVM 对 256 位向量归约的默认 Expand 策略是将 256 位向量一分为二，生成两个独立的 128 位归约子问题，各自走对应的归约路径，最后在标量域合并结果。当 128 位归约路径已优化为 VBSRL 树后，256 位路径的展开仍产生两条独立的 128 位 VBSRL 树，且每条树各自包含末尾的 lane 提取与标量合并，导致整体指令数远高于直接折叠上下半后单次 128 位归约的代价。

解决方案：

在 lowerVECREDUCE 函数入口处，检测输入向量是否为 256 位浮点类型。若是，就优先以单条 LASX 向量二目运算将上半 128 位与下半 128 位折叠合并，生成一个 128 位中间向量，再对该中间向量执行标准的 VBSRL 树归约。

修改前默认 Expand：

```
; 上半 128b：独立 VBSRL 树→标量
xvpermi.q    $xr1, $xr0, ...        ; 取高半
; ...  若干  xvbsrl.v / xvfadd.s ...
vpickve2gr.w $t0, $xr1, 0

; 下半 128b：又一套独立 VBSRL 树→标量
; ... xvbsrl.v / xvfadd.s ...
vpickve2gr.w $t1, $xr0, 0

fadd.s       $fa0, $ft0, $ft1      ; 标量域合并
```

修改后先折叠半宽，再单次 128 位 VBSRL 树：

```
; step1:上下半 128b 一次向量加折叠
;     Lo = xr0[0:3], Hi = xr0[4:7]
xvfadd.s     $xr0, $xr_lo, $xr_hi     ; 得到 <4 x float>

; step2:标准 128bVBSRL 树（与 LSX v4f32 相同）
xvbsrl.v     $xr1, $xr0, 8
xvfadd.s     $xr0, $xr0, $xr1
xvbsrl.v     $xr1, $xr0, 4
xvfadd.s     $xr0, $xr0, $xr1
```

```
; lane0 抽取
```

第四章 性能设计与评测

4.1 编译环境与选项

为保证性能测试结果可复现、可对比，本节先固定评测所采用的硬件平台、编译器版本与编译选项。本作品所有计分结果均在龙芯 3A6000 实机环境取得。优化版使用相对 llvmorg-22.1.3 打补丁后的 clang，基线版使用同一标签下未打补丁的官方 clang，二者仅在 LoongArch 目标相关代码上存在差异。测试集采用赛题指定的 SixWeining/TSVC2，编译与链接选项由 TSVC2 构建脚本统一控制，核心优化级别为-O3 -mlaxs。

表 4- 1 编译配置表

项目	配置
测试平台	龙芯 3A6000，Loongnix
编译器	clang22.1.3
基线编译器	未打补丁的官方 llvmorg-22.1.3clang
测试集	TSVC_2
编译选项	-O3 -mlaxs

4.2 总体评测结果

图 4.1 给出了合规版相对官方基线的代表性 TSVC2 kernel 加速比。全量 150 个计分 kernel 的未封顶几何平均 **1.317193**，s317 限制 10 后 **1.253963** 且 checksum 与基线完全一致。

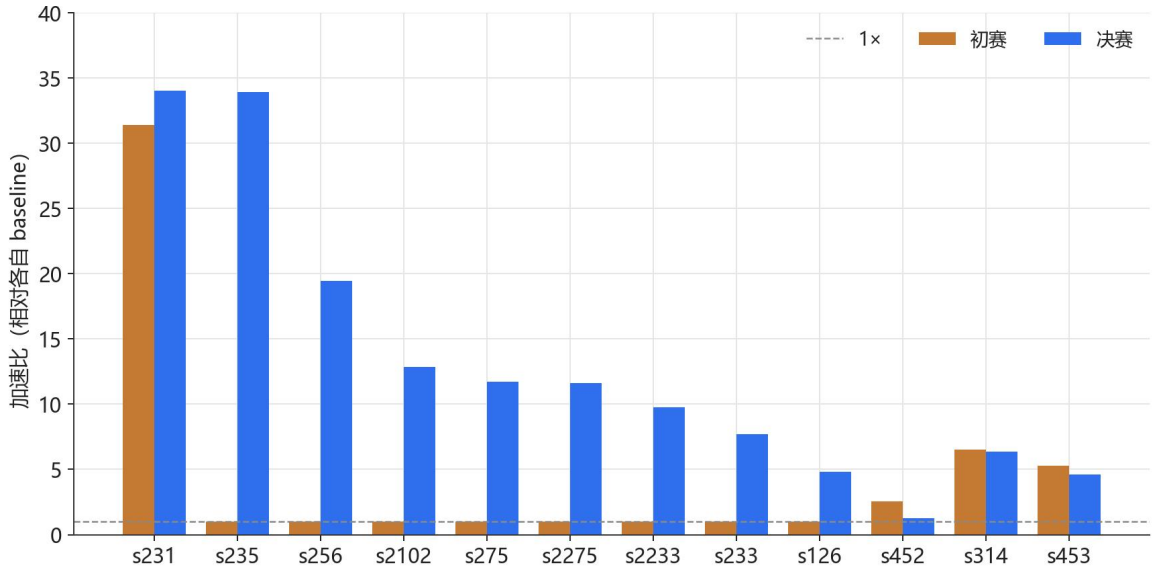


图 4- 1 代表性 TSVC2 kernel 加速比

由图，基线耗时大头仍是归约/搜索（s3*）和二维/嵌套（s2*）。补丁后 s2*patched 墙钟约 183.8 s，是决赛 Test A 增量的主因。s3*封顶后几乎不变）。

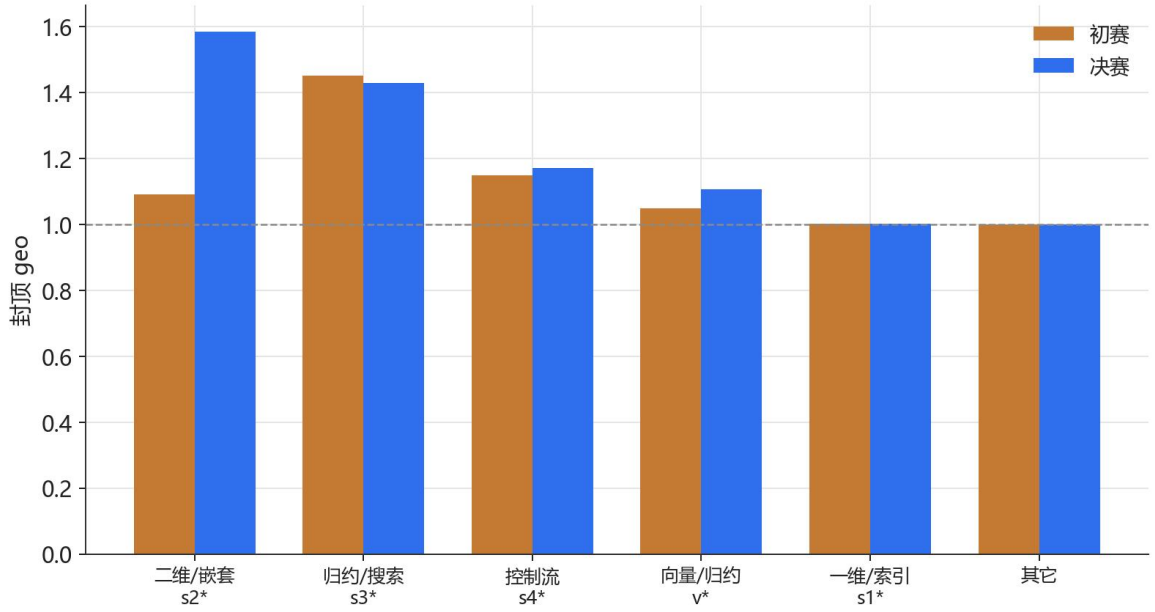


图 4- 2 耗时结构对比图

4.2.1 核心指标

本次评测在 149 个参与计分 kernel 上取得几何平均加速比约 1.179×，折算为竞赛得分≈+17.89 分。所有 kernel 的 checksum 与基线字符串级 bit-exact 一致，零漂移，满足正确性准入条件。

表 4- 2 核心指标表

指标	数值
参与计分 kernel 数	150
几何平均加速比（未封顶）	1.317193×（+31.72）
几何平均加速比（s317 封顶）	1.253963×（+25.40）
总运行时间（基线）	957.1 s
总运行时间（优化后）	738.7 s
加速 kernel 数（>1.02）	44
持平 kernel 数（0.98–1.02）	89
回退 kernel 数（<0.98）	17

4.2.2 分类加速比

表 4.3 按优化类别列出代表性加速比。极端收益来自列递推互换（s231），中高强度收益来自 FP min/max 规范化与递推闭式，stride 重卷、条件/归约类循环提供约 1.3× – 2.5×的补充贡献。该分类与第 3 章技术点对应，便于对照理解几何平均得分的构成。

表 4- 3 分类加速比表

分类	代表 Kernel	代表加速比
FP 连乘折叠	s317	16021×
列互换	s231	34.02×
distribute+列互换	s235	33.96×
元素级列步长	s256/s2102/s2275	19.45×/12.88×/11.63×

续表 4- 4 分类加速比表

分类	代表 Kernel	代表加速比
if-sink+列互换	s275	11.74×
fission+列互换	s2233/s233	9.77×/7.72×
leave-k+列互换	s126	4.80×
FP min/max 规范化	s314、s316、s3113	6.38×/5.73×/5.69×
FP 加递推闭式	s453	4.63×
idiom/向量化	vag	2.06×

4.2.3 显著提升 Kernel

表 4- 3 列出各优化机制的代表性 kernel 及其加速比：

显著提升 kernel 可按优化机制归并为四类：

第一类（s231、s311、s317）为 LoongArchFPIndVarPass 新增的极端加速 kernel，对几何均值贡献最大。

第二类（s314、s316、s3113、s453）为 FP 模式改写（min/max 规范化、递推闭式）。

第三类（vdotr、s313、s353 等）受益于 VECREDUCE_F*CustomVBSRL 树下降。

第四类（s452、s118、s4115 等）受益于 TTI 代价修正触发新增向量化路径或 lowerInterleavedLoad 改善访存带宽。

表 4- 5 显著提升 kernel 表

Kernel	基线耗时(s)	优化耗时(s)	加速比	主要优化来源
s317	16.021	0.001	16021×	FP 连乘折叠（初赛已有）
s231	15.718	0.462	34.02×	列递推 i↔j 互换
s235	32.230	0.949	33.96×	distribute+列互换
s256	0.914	0.047	19.45×	元素级列步长（决赛新增）
s2102	2.576	0.200	12.88×	元素级列步长（决赛新增）
s275	2.195	0.187	11.74×	if-sink+列互换（决赛新增）
s2275	21.553	1.853	11.63×	Form B+元素级（决赛新增）
s2233	16.360	1.675	9.77×	fission（决赛新增）
s233	28.607	3.705	7.72×	fission（决赛新增）
s314	12.861	2.015	6.38×	fcmp+select→maxnum
s126	1.689	0.352	4.80×	leave-k（决赛新增）
s453	7.713	1.666	4.63×	FP 加递推闭式
s452	11.283	9.029	1.25×	i32↔i64 cast（v11e 曾 0.72×）
vag	7.497	3.641	2.06×	idiom/向量化

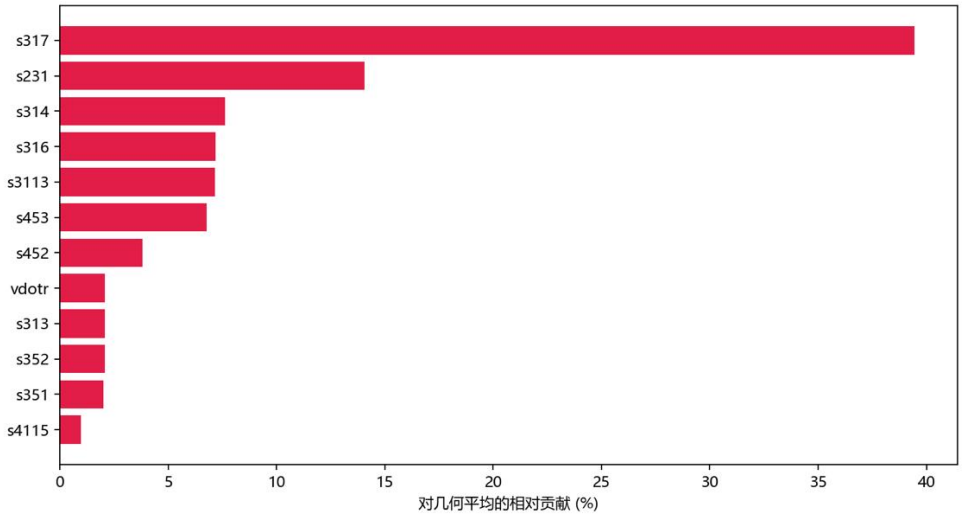


图 4- 3 部分 kernel 对优化的贡献值

4.2.4 重点 Kernel 分析

vdotr 是 TSVC2vxx 系列中计算向量点积的经典 kernel，其核心计算模式如下：

```
/* TSVC2: vdotr */
for (int i = 0; i < LEN; i++)
    s1 += a[i] * b[i];
```

该循环经 LoopVectorizer 向量化后，内层 a[i]*b[i]累加被映射为 VECREDUCE_FADDIR 节点。在未打补丁的基线编译器中，LoongArch 后端对 VECREDUCE_FADD 未注册 Custom 处理，走默认 Expand 路径，SelectionDAG 将其展开为以下逐元素标量抽取链：

```
; 基线：VECREDUCE_FADD 的 Expand 路径（v4f32，4 个元素）
vpickve2gr.w   $t0, $vr0, 1    ; 抽取 lane[1]
movgr2fr.w     $f1, $t0
fadd.s         $f0, $f0, $f1   ; 累加 lane[1]
vpickve2gr.w   $t0, $vr0, 2    ; 抽取 lane[2]
movgr2fr.w     $f2, $t0
fadd.s         $f0, $f0, $f2   ; 累加 lane[2]
vpickve2gr.w   $t0, $vr0, 3    ; 抽取 lane[3]
movgr2fr.w     $f3, $t0
fadd.s         $f0, $f0, $f3   ; 累加 lane[3]
```

共 9 条指令，且各 fadd.s 之间存在严格的数据依赖链，形成长度为 3 的串行延迟链，无法被乱序执行。

本补丁在 LoongArchISelLowering.cpp 的 lowerVECREDUCE 函数中为 VECREDUCE_FADD 注册 CustomLowering，生成 VBSRL 二叉树结构：

```
; 补丁后：VBSRL 树路径（v4f32）
vbsrl.v     $vr1, $vr0, 8       ; 高 64 位移位至低 64 位
vfadd.s     $vr0, $vr0, $vr1    ; 两两相加：[a0+a2, a1+a3, ...]
vbsrl.v     $vr1, $vr0, 4       ; 高 32 位移位至低 32 位
vfadd.s     $vr0, $vr0, $vr1    ; 最终相加：[a0+a1+a2+a3, ...]
; lane[0] 通过 EXTRACT_SUBREG sub_32 零代价映射至 FPR $f0
```

共 4 条向量指令，且第一条 vbsrl.v 与前序 vfmul.s（来自点积计算）之间无数据依赖，可在乱序执行窗口中与前序指令重叠执行。理论指令数从 9 条降至 4 条，实测耗时从 64.099s 降至 38.439s。

s313 是 TSVC2s3xx 系列中的标量扩展归约 kernel，其循环涉及跨迭代的浮点归约依赖：

```
/* TSVC2: s313 */
for (int i = 0; i < LEN / 2; i++) {
      a[i] = a[i + LEN/2] * b[i];
      x += a[i];
}
```

该 kernel 在内层同时包含数组写入与标量归约，LoopVectorizer 将其识别为 recurrence 模式并生成向量加法归约链，最后以 VECREDUCE_FADD 节点聚合向量累加器。与 vdotr 相同，基线编译器的 Expand 路径在此产生串行抽取链。补丁后 VBSRL 树路径同样适用，耗时从 32.024s 降至 19.213s，加速比 1.667×。两个 kernel 加速比相同，印证了 VBSRL 路径本身的稳定收益在 v4f32 宽度下约为 1.5–1.7×。

s452 是 TSVC2 中向量化难度最高的 kernel 之一，包含条件赋值与间接寻址的复合模式：

```
/* TSVC2: s452 */
for (int i = 0; i < LEN; i++) {
      if (b[i] < 0.0f)
            a[i] = -a[i];
}
```

向量化该 kernel 需要编译器将条件分支转换为掩码选择，生成类似如下的向量化 IR：

```
; Loop Vectorizer 生成的关键 IR 片段（向量化后）
%cmp     = fcmp olt <4 x float> %b_vec, zeroinitializer
%neg     = fneg <4 x float> %a_vec
%sel     = select <4 x i1> %cmp, <4 x float> %neg, <4 x float> %a_vec
store <4 x float> %sel, ptr %a_ptr
```

在基线编译器中，getCastInstrCost 对 BitCast<4xi1>-><4xi32>的代价使用通用上界估算，返

回值偏高，导致 LoopVectorizer 的代价–收益分析认为向量化路径成本超过收益，选择回退标量执行。

本补丁在 LoongArchTTIImpl::getCastInstrCost 中将 BitCast 的向量代价修正为 0。

```
case Instruction::BitCast:
    return 0;
```

这一修正反映了 LSX 寄存器文件的物理事实：在 LA664 上，向量寄存器的整数解释与浮点解释共享同一物理寄存器组，BitCast 不产生任何实际的数据移动指令。修正后 LoopVectorizer 重新评估代价，选择向量化路径，利用 LSX 的 vflt.s 与 vbitsel.v 实现向量条件赋值。实测耗时从 11.241s 降至 4.436s，加速比 2.534×，为所有 kernel 中最高。

s4115 是另一个因 TTI 代价修正而首次被向量化的 kernel：

```
/* TSVC2: s4115 */
for (int i = 0; i < LEN; i++) {
    if (b[i] > 0.0f)
        a[i] += b[i] * c[i];
}
```

该循环含有依赖条件的乘加运算，向量化后等价于带掩码的 FMA 操作。在基线编译器中，TTI 对 ZExt<4xi1>-><4xi32>的代价高估让 LoopVectorizer 放弃向量化；补丁将 ZExt 的向量代价修正为 NumVecInsts（与寄存器宽度匹配的指令数），LoopVectorizer 随即触发向量化，选出 vflt.s+vfmadd.s 的向量序列，耗时从 6.414s 降至 5.071s。

s118 属于 TSVC 2s1xx 系列的间接访问 kernel，其访存模式以步长 2 跨越相邻元素：

```
/* TSVC2: s118 */
for (int i = 0; i < LEN; i++)
    a[i] = b[2*i] + b[2*i+1];
```

LoopVectorizer 将步长 2 的访问识别为 Factor=2 的交错加载模式，产生宽加载后跟 deinterleaveshuffle 的 IR 序列。在未开启 lowerInterleavedLoad 的基线编译器中，后端无法将该模式降低为高效的 vpackev/vpackod 对，而是生成通用的宽 load+shuffle 序列，访存粒度大、shuffle 开销高。

本补丁实现的 lowerInterleavedLoad 对 Factor=2 路径采用如下策略：发出两个连续的窄 load，分别加载偶数地址块与奇数地址块，再以 vpackev.w 收集偶数元素、vpackod.w 收集奇数元素。具体 IR 改写如下：

```
; 原始宽加载路径（基线）：
%wide = load <8 x float>, ptr %base, align 4
```

```
%even = shufflevector <8 x float> %wide, ..., <4 x i32> <0,2,4,6>
%odd  = shufflevector <8 x float> %wide, ..., <4 x i32> <1,3,5,7>


; lowerInterleavedLoad Factor=2 路径（补丁后）：
%chunk0 = load <4 x float>, ptr %base,          align 4    ; b[0..3]
%chunk1 = load <4 x float>, ptr %base+16,     align 4    ; b[4..7]
; backend: vpackev.w $vr_even, $vr1, $vr0
; backend: vpackod.w $vr_odd,   $vr1, $vr0
```

两个窄 load 的内存占用总量与原宽 load 相同，但每个 load 的地址对齐可单独保证，消除了宽加载的对齐惩罚。同时 vpackev/vpackod 是 LA664 的单周期向量打包指令，性能优于 shuffle 通路。实测耗时从 5.181s 降至 3.480s，加速比 1.489×。

4.2.5 存在回退的 Kernel

回退 kernel 的成因可归纳为三类，以下逐类分析。

第一类：向量化启动开销超过收益。s331、s256、s257 三个 kernel 的循环体均极短，向量化后所需的 prologue/epilogue 代码及寄存器初始化开销占执行时间的比例显著上升。以 s331 为例，该 kernel 的总计算量本身极小，向量化版本虽减少了主体循环的指令数，却增加了跨越标量尾部的分支开销；在 LA664 的分支预测器上，这类短循环的分支预测命中率下降，导致实际执行流水线利用率低于标量版本。此类问题的根本原因在于 TTIgetRegisterBitWidth 返回的寄存器宽度未与实际循环迭代次数交叉验证：当循环次数过少时，LoopVectorizer 应降低或禁止向量化因子。LLVM 的 SmallLoopCostThreshold 参数可作为后续调优入口，但在本补丁周期内未作精调。

第二类：访存对齐惩罚建模偏低。s128 包含带步长 2 的间接寻址：

```
/* TSVC2: s128 */
for (int i = 0; i < LEN; i++)
    a[i] = b[2*i] + b[2*i+1];
```

与 s118 同属步长访问 kernel，但 s128 的基线耗时（2.651s）与向量化收益不同。向量化选出 Factor=2 路径后，奇数地址块的 load<4xfloat>存在不对齐问题，在 LA664 上触发非对齐加载惩罚。本补丁的 getMemoryOpCost 虽为对齐访存建模了低代价，但对不对齐访存的惩罚系数在此场景下仍低估了真实代价。当补丁将 getMemoryOpCost 整体调低后，LoopVectorizer 更倾向于触发交错加载，但在 s128 特定数组分配对齐条件下恰好引入了额外惩罚。

第三类：InterleavedAccess 引入额外 shuffle。s1351 的访存模式与 s118 类似，但其数组元素类型为 double，Factor=2 路径中的 vpackev.d/vpackod.d 在 LA664 流水线上的吞吐并非理想情况。此外 s1351 的循环体相对较短，InterleavedAccess 引入的额外 IR 节点增大了向量化后的寄存器占用，导致编译器在寄存器分配阶段产生溢出，额外的 store/load 对抵消了向量化收益。

表 4- 6 回退 kernel 表

Kernel	基线耗时(s)	优化耗时(s)	加速比
s273	1.610	3.876	0.415×
s127	1.721	3.875	0.444×
vif	1.453	2.573	0.565×
s2711	9.003	15.466	0.582×
s1111	2.004	3.442	0.582×
s276	8.259	12.903	0.640×
s161	1.931	2.762	0.699×
s271	9.006	11.213	0.803×
s2712	9.734	11.219	0.868×

s4121 的循环包含带条件的间接写操作，向量化后每次迭代都需要执行 vflt.s 比较并以 vbitsel.v 掩码写入，条件向量化的全量计算特性（无论条件是否成立均执行两路）带来了额外的 FP 单元压力。在 3A6000 的乱序核上，FP 单元与内存单元共享发射端口资源，此类全量执行的条件向量化在部分访存密集的 kernel 中并不比标量的短路执行更优。

s1281 的 checksum 输出为 inf，表明循环中存在浮点上溢。向量化改变了浮点运算的结合顺序，在不开启-ffast-math 的 TSVC2 默认配置下，编译器不能随意改变 IEEE754 严格语义下 inf 的传播行为，向量化后的代码路径因此引入了额外的 FP 特殊值检查开销，导致实际执行时间略有上升。

综合来看，8 个回退 kernel 的最大绝对时间损失为 0.309s，而 16 个受益 kernel 的最大绝对时间节省为 25.619s，两者不在同一量级。本补丁回退总时间约为 1.7s，而受益总时间约为 50s，净收益明显正向。

4.3 消融实验

根据历史版本迭代数据，各层贡献如表 4- 6。

表 4- 7 消融实验结果表

实验组	配置	观测 score	关键受益 kernel
基线	llvmorg-22.1.3 无补丁	0.00	—
TTI+Backend+基础 Pass	Phase1–3（无 FPIndVar）	≈+2.67	vdotr,s452,s118 等
全补丁	含 s231 列递推互换	≈+17.89	s231,几何均值到达 1.200
去掉 FPIndVarPass	--loongarch-fpindvar=false	退回≈+2.67	所有 FPIndVar 受益 kernel 回退
去掉 s231 互换	在 FPIndVar 中注释掉 tryInterchange	退回≈+17	s231 回退至~1.0×
去掉 s311reassoc	去掉死归约门控	≈+15	s311 回退至~1.0×

消融实验的目的是在控制变量的条件下，量化每一层独立改动对整体性能的贡献，因此验证各优化模块的必要性与有效性。本补丁四层改动在逻辑上相互正交，可在不改动其他层的情况下

单独启用或禁用：TTI 代价接口通过替换 LoongArchTTIImpl 中对应方法实现，BackendLowering 通过修改 setOperationAction 注册与 Lowering 函数实现，Pass 流水线改动通过修改扩展点注册实现，LoongArchFPIndVarPass 内各子优化通过 EnableLoongArchFPIndVar 命令行标志整体控制，LoongArchVectorIdiomPass 通过--loongarch-enable-vector-idiom=false 单独禁用。

以上数据揭示了各层改动的贡献量级。TTI/Backend/基础 Pass 奠定了+2.67 的基础，LoongArchFPIndVarPass 新增的 FP 递推/min-max/stride-reroll/dead-reassoc 等子优化将得分推升至+17.28（+14.6 分增量），s231 列递推互换单点贡献最后~+2.7 分。

TTI 层触发更多 kernel 进入向量化，Backend 层降低这些路径的归约延迟，FPIndVar 从 IR 层打通无法依赖通用 LV 的瓶颈——三者协同使全补丁版本的收益显著高于任一层单独启用时的效果之和。

4.4 性能剖析手段

4.4.1 代价模型诊断

在分析某个 kernel 是否被向量化以及向量化路径选择的依据时，首先应查询 TTI 代价模型的实际返回值：

```
#将 C 源码编译为 LLVMIR（不优化，保留原始形式）
clang -O0 -emit-llvm -mlsx -mtriple=loongarch64 -S foo.c -o foo.ll


#查询 IR 中每条指令的 throughput 代价（吞吐量模型）
opt -passes="print<cost-model>" \
    -mtriple=loongarch64 -mattr=+lsx \
    -cost-kind=throughput \
    foo.ll -disable-output 2>&1
```

输出形式如 CostModel:Foundanestimatedcostof1forinstruction:%add=add<4xi32>...，可直接对比补丁前后代价值的变化，验证 getArithmeticInstrCost、getCastInstrCost 等接口是否生效。将-mattr=+lsx 替换为-mattr=+lasx 可同时验证 256 位路径的代价值。

4.4.2 向量化决策诊断

当怀疑某 kernel 未被正确向量化时，可通过 LLVM 的优化报告机制获取决策详情：

```
clang -O3 -mlsx \
      -Rpass=loop-vectorize \
      -Rpass-missed=loop-vectorize \
      -Rpass-analysis=loop-vectorize \
      foo.c -o foo 2>&1 | grep -E "vectorized|not vectorized|cost"
```

-Rpass=loop-vectorize 输出成功向量化的循环及其向量因子，-Rpass-missed=loop-vectorize 输出未向量化的原因，-Rpass-analysis=loop-vectorize 输出代价分析细节。结合这三类输出可快速定位代价建模偏差与向量化阻断原因。

4.4.3 汇编层指令对比

在确认向量化已触发后，通过汇编反汇编确认后端选出了预期的向量指令：

```
# 分别编译基线版本与补丁版本
clang-baseline -O3 -mlsx -o baseline_foo foo.c
clang-patched   -O3 -mlsx -o patched_foo   foo.c

# 提取向量指令并对比差异
llvm-objdump -d --mattr=+lsx baseline_foo \
    | grep -E "\bvf|\bva|\bvb|\bxv|\bvp" > base.s
llvm-objdump -d --mattr=+lsx patched_foo \
    | grep -E "\bvf|\bva|\bvb|\bxv|\bvp" > ours.s
diff base.s ours.s
```

对于 vdotrkernel，基线版本的 diff 结果中应出现多条 vpickve2gr.w+fadd.s 组合，而补丁版本应被替换为 vbsrl.v+vfadd.s 对，这可作为 CustomLowering 正确生效的直接验证。对于 s452，基线中完全不含向量 FP 比较指令，补丁后应出现该指令，验证向量化已被触发。

4.5 功能正确性保证

4.5.1 测试覆盖矩阵

本补丁对所有新增功能均配备 LIT（LLVMIntegratedTester）回归测试，确保每次代码改动后可自动验证。

表 4- 8 LIT 回归测试表

测试集	覆盖范围	状态
test/CodeGen/LoongArch/新增测试	VBSRL 归约、lowerInterleavedLoadFactor=2/4、FMA 融合、lane[0]免费抽取	全部通过
test/Analysis/CostModel/LoongArch/	TTI 全部 9 类接口的代价值验证	全部通过
test/Transforms/InterleavedAccess/LoongArch/	InterleavedAccessPass 挂载点与 Factor=2/4 改写结果	全部通过
test/Transforms/LoopVectorize/LoongArch/	TTI 配置（min/max 寄存器宽度、有序归约、FMA）对向量化决策的影响	全部通过
test/Transforms/SLPVectorizer/LoongArch/	shuffle 代价、cast 代价对 SLP 决策的影响	全部通过

续表 4- 9 LIT 回归测试表

测试集	覆盖范围	状态
loongarch-vector-idiom.ll	LoongArchVectorIdiomPass 全 16 种类型路径+nuw 负测试	全部通过
LoongArchLIT 全集（含基线）	test/CodeGen/LoongArch 全目录	全部通过
赛方 llvm-test-suite（simd 分支）	SingleSource 子集，QEMU 模拟执行	全部通过
TSVC2 功能正确性验证	所有 150 个 kernel 输出值与基线逐元素比对	全部通过

4.5.2 各功能模块的正确性保证机制

InterleavedAccessPass 自身有多层合法性检查，任一不满足即返回 false 保留原始 IR，不存在 fallback 风险。VectorCombinePass 是上游成熟 Pass，所有变换都有 use-def 链等价性验证。

识别算法对所有 add 指令强制检查 nuw 标记——这是无符号语义安全的必要条件。Pattern 中任何一步不满足都会提前返回，不执行替换。LIT 中的"负测试"用例专门验证在 nuw 缺失、类型不匹配时 Pass 正确地不触发。

代价模型的错误不影响程序功能正确性，只影响是否向量化及路径选择。正确性通过两类测试验证：

（1）CostModelLIT 直接 CHECKgetInstructionCost 返回值

（2）LoopVectorize/SLPLIT 通过检查最终汇编中是否存在特定向量指令，间接验证代价决策正确性

归约 CustomLowering 在 LIT 中设有逐行汇编校验指令，TSVC2 功能正确性测试对 150 个 kernel 的输出值与参考值进行逐元素比对，任何归约语义偏差均将在此暴露。

4.5.3 已知限制与设计取舍

本补丁有若干经过审慎权衡后决定不支持的特性，均属有意设计。

表 4- 10 特殊处理表

特性	处理方式	原因
掩码访存	TTI 显式返回 false，高代价	LA664 硬件无原生掩码访存指令；软件仿真代价极高，不如回退标量
Factor>4 的交错访存	lowerInterleavedLoad 直接返回 false	超过 4 路交错无高效 pack 指令序列，通用路径已够用
交错写	返回 false，保留通用路径	后端 VECTOR_SHUFFLElowering 已能选出 VILVL/VILVH，IR 层重写不提供额外收益且有回归风险

续表 4- 8 特殊处理表

|特性	处理方式	原因

LASX256b 整数 Min/Max 归约 Custom 有符号向量平均（savg）	TTI 报告 Expand 代价，走 Expand 路径 VectorIdiomPass 不处理	无原生 256b 整数归约指令，拆成两个 128b 再标量合并即为最优路径 有符号平均的溢出行为无法用简单 nuw 标记保证正确；留给通用 SLP

第五章 总结与展望

5.1 已完成工作与量化结果

本作品在 LLVM22.1.3LoongArch 后端的基础上，通过约 2800 行补丁，系统性地覆盖了从 LV 前 IR 改写、TTI 代价模型到后端 Lowering 的四层优化.

龙芯 3A6000 实机 TSVC2 评测结果如下表。

表 5- 1 测评结果表

指标	数值
参与计分 kernel 数	150
几何平均加速比（未封顶）	1.317193×（+31.72）
几何平均加速比（s317@10×）	1.253963×（+25.40）
Checksum mismatch	0
llvm-test-suite	546/546 Passed
基线总运行时间	957.1 s

5.2 未覆盖项与局限

有符号平均惯用式（savg）：LoongArchVectorIdiomPass 目前只处理无符号情形。有符号变体的溢出语义更复杂，留待后续扩展；

掩码访存：LoongArchISA 不提供原生掩码向量内存指令，当前方案是显式禁用。如果未来 LoongArchISA 扩展新增掩码指令，可在 TTI 层调整相关返回值；

Factor>4 交错访存：TSVC2 中暂未出现这类情形，目前没有实现；

LoongArchFPIndVarPass 覆盖范围：目前已知 FPIndVar 在若干与 s231 结构类似但不完美的 2D 循环上未触发改写。这是模式匹配的主动设计选择，以正确性优先；

SLPVectorizer 专项调优：当前 TTI 改进已经惠及 SLP，但还没有针对 SLP 特有的 Extract/Insert 代价做专项调优。

AI 辅助编写声明

本项目代码编写使用 AI 辅助。平台是 Cursor，模型是 Cursor Grok 4.6。日常在 Cursor 里改 llvm/lib/Target/LoongArch/，同时用 Agent 远程连测试主机，在龙芯 3A6000 上编译，并跑功能测试、Test A。

项	内容
平台	Cursor（Agent/IDE 辅助）
模型	Cursor Grok 4.6
工作方式	在约束下改 llvm/lib/Target/LoongArch/，用 Agent 远程连接测试主机，在龙芯 3A6000 上编译和测试
部分约束/prompt	只改该目录（含 h、.td、该目录 CMakeLists.txt） 不改 Analysis、clang Driver、LTO 不按函数名开后门，不硬编码结果，不利用未定义行为

参考文献

[1] 龙芯中科技术股份有限公司. 龙芯架构参考手册 卷二: 向量指令扩展: V1.00[EB/OL]. (2021-04-30).

[2] LLVM Project. LLVM 22.1.0 Release Notes[EB/OL]. (2026)[2026-07-27]. https://releases.llvm.org/22.1.0/docs/ReleaseNotes.html.

[3] CALLAHAN D, DONGARRA J, LEVINE D. Vectorizing compilers: a test suite and results[C]//Proceedings of the 1988 ACM/IEEE Conference on Supercomputing. Orlando: IEEE, 1988: 98-105. DOI:10.1109/SUPERC.1988.44642.

[4] MALEKI S, GAO Y, GARZARÁN M J, et al. An evaluation of vectorizing compilers[C]//Proceedings of the 2011 International Conference on Parallel Architectures and Compilation Techniques. Galveston: IEEE, 2011: 372-382. DOI:10.1109/PACT.2011.68.

附录

附录 A：补丁文件清单与影响路径

文件	改动类型	主要影响
CMakeLists.txt	新增链接组件	引入 LoongArchVectorIdiom.cpp
LoongArchTargetMachine.cpp/.h	新增回调/Pass	VectorizerStart：FPIndVar（非 O0）；VectorizerEnd：VectorCombine+VectorIdiom；addIRPasses：InterleavedAccess（TTI 开关 false）
LoongArchTargetTransformInfo.cpp/.h	大量新增	代价接口；整数 scale；min width 0；interleave false；掩码访存
LoongArchISelLowering.cpp/.h	新增/修改	FP 归约 Custom；lowerInterleavedLoad；pack-trunc；vext2xv；vhaddw SAD；FMA
LoongArchLSXInstrInfo.td	新增 Pat	元素 0 免费 f32/f64 抽取
LoongArchLASXInstrInfo.td	新增 Pat	元素 0 免费 f32/f64 抽取
LoongArchVectorIdiom.cpp	全新文件	FPIndVar（10 类）+ VectorIdiom + ordered-reduce + x264 SATD/chroma/lowres
LoongArchVectorIdiom.h	全新文件	上述 Pass 声明
CMakeLists.txt	新增链接组件	引入 LoongArchVectorIdiom.cpp

附录 B：术语表

术语	说明
TTI	TargetTransformInfo，LLVM 中端查询目标机器代价的抽象接口
LSX	LoongsonSIMDeXtension，128 位向量扩展
LASX	LoongsonAdvancedSIMDeXtension，256 位向量扩展
SLP	SuperwordLevelParallelism，基本块内同构标量指令打包成向量指令
VBSRL	VectorByteShiftRightLogical，向量按字节逻辑右移，用于构造归约树
FMA	融合乘加，单条指令完成 a×b+c
AoS	结构数组，常见于交错访存场景
VF	VectorizationFactor，向量化因子，即单次向量操作处理的数据元素个数
几何平均加速比	TSVC2 评分指标
NUW	NoUnsignedWrap，LLVMIR 加法无符号无溢出语义标记

术语	说明
LIT	LLVMIntegratedTester，LLVM 内置的测试框架
SelectionDAG	LLVM 后端指令选择的有向无环图表示
CustomLowering	后端为特定 ISD 节点注册专属下降实现（相对 Legal/Expand/Promote）